POSTGRESQL ON KUBERNETES

Operations & Troubleshooting Runbook v2.0

Platform	SS&C Cloud — VM-based Kubernetes (Alma Linux 9 nodes)
Operator	Percona PostgreSQL Operator v2.x (PerconaPGCluster CRD)
HA Stack	Patroni (cluster mgmt) + HAProxy (load balancing) + PGBouncer (pooling)
Storage	CSI Driver with VM-attached disks; PersistentVolumeClaims
Backup	pgBackRest — WAL archiving + full/differential/incremental backups
Monitoring	Prometheus + Alertmanager + postgres_exporter + Grafana
Document Date	2026

Severity Classification

CRITICAL	Service down; data at risk; page on-call immediately; P1 incident.
HIGH	Risk of service impact within hours; resolve immediately; P2 incident.
MEDIUM	Degraded performance or latency; resolve within 24 h; P3 incident.

LOW	Minor / cosmetic; schedule for next business day; P4 ticket.

Quick Reference Index — All 30 Issues

#	Issue Title	Severity	Component
01	Patroni Failover Not Triggering	CRITICAL	Patroni / etcd
02	PVC / CSI Mount Failure	CRITICAL	Storage
03	PGBouncer Pool Exhaustion	HIGH	PGBouncer
04	HAProxy Health Check Failures	HIGH	HAProxy
05	High Replication Lag	HIGH	Replication
06	Pod OOMKilled / Memory Pressure	CRITICAL	Resources
07	Disk Space Exhaustion on PV	CRITICAL	Storage
08	Operator CRD Reconciliation Errors	HIGH	Percona Operator
09	Slow Queries / Lock Contention	MEDIUM	Query Engine
10	WAL Archiving / pgBackRest Failure	HIGH	Backup
11	Certificate Expiry (TLS)	HIGH	Security
12	Node NotReady (VM Failure)	CRITICAL	Infrastructure
13	autovacuum / Table Bloat	MEDIUM	Maintenance
14	PGBouncer Authentication Failures	MEDIUM	Authentication
15	Prometheus Metrics Not Scraping	MEDIUM	Monitoring
16	Pod CrashLoopBackOff	CRITICAL	Runtime
17	Resource Quota Blocking Startup	MEDIUM	Scheduling
18	DNS / Service Discovery Failures	MEDIUM	DNS
19	Rolling Upgrade Failure	HIGH	Lifecycle
20	Logical Replication Failure	HIGH	Replication
21	Transaction ID Wraparound Emergency	CRITICAL	MVCC / Safety
22	Split-Brain After Network Partition	CRITICAL	HA / Data

			Integrity
23	pgBackRest Restore Hung / Slow	**HIGH**	Backup & Recovery
24	StatefulSet VCT Changes Blocked	**MEDIUM**	Storage Config
25	High WAL Generation / I/O Saturation	**HIGH**	Storage I/O
26	Secret Drift / Config Out of Sync	**MEDIUM**	Configuration
27	max_connections Exhaustion	**HIGH**	Connection Mgmt
28	Node Affinity / Taint Scheduling Failure	**MEDIUM**	Scheduling
29	Index Corruption Detected	**CRITICAL**	Data Integrity
30	Cross-Namespace NetworkPolicy Denied	**MEDIUM**	Network Security

Detailed Runbooks — Issues 01 to 20

ISSUE 01 — Patroni Failover Not Triggering / Stuck in Leader Election

CRITICAL

Component	Patroni / Percona Operator
Affected Layer	HA Cluster Control Plane
Common Cause	DCS (etcd) unavailability, network partition, clock skew between VMs
Alert Name	patroni_master_is_running == 0 for > 60s
Recovery Time	2–10 min automated; up to 30 min manual

Symptoms

- kubectl get pods shows all Patroni pods running but no leader promoted.
- patronictl list shows all nodes as 'Replica' or blank leader field.
- HAProxy health checks failing on /primary endpoint — all backends marked DOWN.
- Applications receive 'could not connect to the server' errors.
- Operator logs show repeated reconcile errors for PerconaPGCluster.
- Prometheus alert PatroniNoLeader firing for more than 1 minute.

⌨ DIAGNOSTIC COMMANDS

```
# — Patroni cluster state
```

Cluster overview	`kubectl exec -n <ns> <patroni-pod> -- patronictl -c /etc/patroni/patroni.yml list`
Topology view	`kubectl exec -n <ns> <patroni-pod> -- patronictl -c /etc/patroni/patroni.yml topology`
Switchover history	`kubectl exec -n <ns> <patroni-pod> -- patronictl -c /etc/patroni/patroni.yml history`

```
# — etcd DCS health
```

etcd member health	`kubectl exec -n <ns> <etcd-pod> -- etcdctl endpoint health --cluster`
etcd member list	`kubectl exec -n <ns> <etcd-pod> -- etcdctl member list`

```
# ── Clock skew check on Alma9 VM nodes ──────────────────────
```

NTP sync status	`ssh <node-ip> 'chronyc tracking | grep -E "System time|Stratum"'`
Time diff between nodes	`for n in node1 node2 node3; do echo "$n:"; ssh $n date; done`

```
# ── Operator and pod logs ────────────────────────────────────
```

Operator logs	`kubectl logs -n <ns> deploy/percona-postgresql-operator --tail=200`
Patroni pod logs	`kubectl logs -n <ns> <patroni-pod> -c patroni --tail=200`
Cluster events	`kubectl get events -n <ns> --sort-by=.metadata.creationTimestamp | tail -40`

> ℹ️ **NOTE**
> Clock skew > 500 ms between Kubernetes/VM nodes will cause etcd Raft consensus failures and prevent leader election. Always verify chronyc tracking shows System time offset < 100 ms on all Alma9 nodes before investigating elsewhere.

Resolution Steps

1	Verify etcd cluster health — all 3 or 5 members must be healthy before Patroni can elect a leader.
2	Fix NTP on affected VM nodes: systemctl restart chronyd && chronyc makestep — then re-verify chronyc tracking.
3	If etcd quorum is lost, restore from the most recent snapshot: etcdctl snapshot restore <snap.db> --data-dir=/var/lib/etcd-restore.

4	If Patroni is in split-brain, demote the rogue leader: patronictl -c /etc/patroni/patroni.yml demote <node>.
5	Reinitialize a broken standby: patronictl -c /etc/patroni/patroni.yml reinit <cluster> <node> --force.
6	Trigger controlled switchover to healthy replica: patronictl switchover <cluster> --master <old> --candidate <new> --force.
7	Confirm operator reconciles successfully: kubectl describe perconapgcluster -n <ns> | grep -A5 Status.
8	Validate HAProxy picks up new leader within 10 s: curl http://<haproxy-svc>:8008/primary should return HTTP 200.

Prevention & Alerting

- Prometheus rule: max(patroni_master_is_running) by (cluster) == 0 for 60s → CRITICAL.
- Ensure Chrony is configured on all Alma9 nodes and time offset < 100 ms.
- Use odd etcd member count (3 or 5) to maintain quorum during VM failures.
- Patroni tuning: ttl=30, loop_wait=10, retry_timeout=10 for fast failover.
- Schedule monthly failover drills: patronictl switchover --force in a staging namespace.

ISSUE 02 — PVC / PersistentVolume Mount Failure (CSI Driver) — CRITICAL

Component	Kubernetes Storage / CSI Driver
Affected Layer	Storage / Pod Startup
Common Cause	CSI driver crash, stale VolumeAttachment, disk detach at hypervisor level
Alert Name	KubePersistentVolumeFillingUp, PodNotReady

Recovery Time	15–45 min depending on disk state

Symptoms

- PostgreSQL pod stuck in 'ContainerCreating' or 'Init:0/1' for more than 5 minutes.
- kubectl describe pod shows: 'Unable to attach or mount volumes: timed out waiting for the condition'.
- VolumeAttachment object exists but attachment status shows false.
- CSI node plugin pod logs show dial timeout or gRPC connection errors.
- Disk physically present on Alma9 VM but not visible inside the pod.

⌨ DIAGNOSTIC COMMANDS

```
# — Pod and PVC state
```

Describe stuck pod	`kubectl describe pod -n <ns> <pg-pod>`
PVC status	`kubectl get pvc -n <ns>`
PV detail	`kubectl describe pv <pv-name>`

```
# — VolumeAttachment troubleshooting
```

List VolumeAttachments	`kubectl get volumeattachment | grep <pv-name>`
Describe attachment	`kubectl describe volumeattachment <va-name>`

```
# — CSI driver status
```

CSI node pods on node	`kubectl get pod -n kube-system -l app=csi-node -o wide | grep <node-name>`
CSI node driver logs	`kubectl logs -n kube-system <csi-node-pod> -c csi-driver --tail=100`

# — Disk check on Alma9 VM	
Block devices on VM	`ssh <node-ip> 'lsblk'`
Kernel disk messages	`ssh <node-ip> 'dmesg | tail -30'`
Mount table	`ssh <node-ip> 'df -h && mount | grep /dev/sd'`

Resolution Steps

1	SSH into the Alma9 VM node and run lsblk to confirm whether the disk is visible at the OS level.
2	If disk is visible but not mounted inside pod: run 'udevadm trigger' on the node to re-detect the device.
3	Delete the stale VolumeAttachment: kubectl delete volumeattachment <va-name> — Kubernetes will recreate it and reattempt attachment.
4	If the CSI node plugin is crashed, restart the DaemonSet: kubectl rollout restart daemonset/<csi-node-ds> -n kube-system.
5	If the PVC is bound to a dead node, cordon the node then delete the pod to reschedule: kubectl cordon <node> && kubectl delete pod <pg-pod> -n <ns>.
6	For persistent detach failures, force-unmount at OS level on the old node: umount -f /var/lib/kubelet/pods/<uid>/volumes/... then delete the VolumeAttachment.
7	After recovery, verify data integrity: kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c 'SELECT pg_postmaster_start_time();'

Prevention

- Pin CSI driver to a tested release; configure automatic restart policy on CSI pods.
- Prometheus alert: kube_persistentvolumeclaim_status_phase{phase='Lost'} == 1 → CRITICAL.
- Use PodDisruptionBudgets to prevent simultaneous rescheduling of multiple Postgres pods.

- Set StorageClass reclaimPolicy: Retain to prevent accidental data loss on PVC deletion.

ISSUE 03 — PGBouncer Connection Pool Exhaustion — HIGH

Component	PGBouncer
Affected Layer	Connection Management
Common Cause	Long-running/idle transactions, pool_size misconfiguration, connection leaks in app code
Alert Name	pgbouncer_pool_waiting_clients > 50
Recovery Time	5–15 min

Symptoms

- Applications receive 'FATAL: remaining connection slots are reserved' or connection timeout errors.
- PGBouncer SHOW POOLS shows cl_waiting > 0 and maxwait continuously increasing.
- pg_stat_activity on the primary shows connection count near max_connections.
- New application instances cannot connect while existing sessions are fine.
- Prometheus: pgbouncer_pool_waiting_clients_total rising and not recovering.

⌨ DIAGNOSTIC COMMANDS

```
# — PGBouncer admin console (connect via kubectl exec)
```

Connect to admin	`kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U pgbouncer pgbouncer`
Pool status	`kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U pgbouncer pgbouncer -c 'SHOW POOLS;'`
Client list	`kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U`

	`pgbouncer pgbouncer -c 'SHOW CLIENTS;'`
Server connections	`kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U pgbouncer pgbouncer -c 'SHOW SERVERS;'`
Stats per DB	`kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U pgbouncer pgbouncer -c 'SHOW STATS;'`

```
# ── PostgreSQL connection analysis ──────────────────────────
```

Connection count by state	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT count(*), state FROM pg_stat_activity GROUP BY state;"`
Idle-in-transaction sessions	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT pid, now()-xact_start AS duration, application_name, query \ FROM pg_stat_activity WHERE state = 'idle in transaction' \ ORDER BY duration DESC LIMIT 20;"`
Oldest transaction age	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT max(now()-xact_start) FROM pg_stat_activity WHERE state != 'idle';"`

> ⚠ **WARNING**
> Do NOT blindly terminate all connections. Identify the root cause first. Killing legitimate transactions can cause data inconsistency or application errors.

Resolution Steps

1	Run SHOW POOLS in PGBouncer admin — identify which database/user pair has the most cl_waiting clients.
2	Kill idle-in-transaction sessions older than 5 minutes: SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE state='idle in transaction' AND now()-xact_start > interval '5 minutes';
3	Pause the saturated database in PGBouncer to queue new connections gracefully: PAUSE <dbname>; in admin console.
4	Temporarily increase server_pool_size in the PGBouncer ConfigMap and hot-reload: RELOAD; in PGBouncer admin.
5	Resume the database: RESUME <dbname>; — verify cl_waiting drops to 0.

6	Set idle_transaction_timeout = 60 in pgbouncer.ini to auto-terminate future stale transactions.
7	Investigate application code for connection leak patterns — look for missing connection.close() calls.

Recommended PGBouncer Configuration

# [pgbouncer] section — transaction pooling for OLTP workloads	
Pooling mode	pool_mode = transaction
Max client connections	max_client_conn = 5000
Pool size per pair	default_pool_size = 25
Reserve pool	reserve_pool_size = 10 ; fallback for burst
Hard DB cap	max_db_connections = 100 ; total to PostgreSQL
Kill idle transactions	idle_transaction_timeout = 60
Auth method	auth_type = scram-sha-256

ISSUE 04 — HAProxy Backend Health Check Failures — HIGH

Component	HAProxy
Affected Layer	Load Balancer / Traffic Routing
Common Cause	Patroni REST API down, NetworkPolicy blocking port 8008, stale pod IPs in HAProxy config
Alert Name	haproxy_backend_active_servers == 0
Recovery Time	5–20 min

Symptoms

- All application connections are refused or routed to wrong backend.
- HAProxy stats page shows all backends in DOWN or DRAIN state.
- Patroni REST health endpoint (port 8008) not responding from the HAProxy pod.
- kubectl get endpoints shows empty endpoint list for HAProxy service.
- Prometheus alert HAProxyBackendDown firing.

⌨ DIAGNOSTIC COMMANDS

# — HAProxy stats and config	
Backend stats via socket	`kubectl exec -n <ns> <haproxy-pod> -- sh -c \ 'echo "show stat" | socat stdio /var/run/haproxy/admin.sock | cut -d, -f1,2,18,19,20'`
Validate HAProxy config	`kubectl exec -n <ns> <haproxy-pod> -- haproxy -c -f /etc/haproxy/haproxy.cfg`
HAProxy logs (last 100)	`kubectl logs -n <ns> <haproxy-pod> --tail=100`
# — Test Patroni health endpoints from HAProxy pod	
Test primary endpoint	`kubectl exec -n <ns> <haproxy-pod> -- curl -sv http://<patroni-pod-ip>:8008/primary`
Test replica endpoint	`kubectl exec -n <ns> <haproxy-pod> -- curl -sv http://<patroni-pod-ip>:8008/replica`
Test /health endpoint	`kubectl exec -n <ns> <haproxy-pod> -- curl -sv http://<patroni-pod-ip>:8008/health`
# — Network policy and service	
List network policies	`kubectl get networkpolicy -n <ns>`
Check service	`kubectl get endpoints -n <ns> <pg-service>`

endpoints	
Check pod labels	`kubectl get pod -n <ns> --show-labels | grep postgres`

Resolution Steps

1	Access HAProxy stats (port 1936) or admin socket to identify which specific backends are DOWN.
2	From the HAProxy pod, curl the Patroni /primary endpoint on port 8008 for each PostgreSQL pod IP. HTTP 200 = healthy; HTTP 503 = not primary.
3	If Patroni endpoints are unreachable, check NetworkPolicy: must allow TCP ingress on port 8008 from HAProxy pods to PostgreSQL pods.
4	If HAProxy config has stale pod IPs (not using DNS service names), update the ConfigMap backend section to use Kubernetes service DNS names instead of IPs.
5	Reload HAProxy config without downtime: kubectl exec -n <ns> <haproxy-pod> -- haproxy -sf $(cat /var/run/haproxy.pid) -f /etc/haproxy/haproxy.cfg.
6	If the HAProxy pod itself is crashed, delete it and let the Deployment restart it: kubectl delete pod <haproxy-pod> -n <ns>.
7	Verify recovery: all primary backends should show UP in stats and curl /primary should return 200.

ISSUE 05 — High Replication Lag on Standby — HIGH

Component	PostgreSQL Streaming Replication / Patroni
Affected Layer	Data Replication
Common Cause	Network saturation, disk I/O bottleneck, long autovacuum,

	WAL receiver crash, inactive slots
Alert Name	pg_replication_lag > 30s, pg_replication_slot_inactive
Recovery Time	10–60 min depending on cause

Real-World Scenario Runbooks — Issues 21 to 30

The following issues are based on real incident patterns observed in production PostgreSQL deployments on Kubernetes. Each issue includes the specific real-world context that triggered the incident, making them directly applicable to production troubleshooting.

ISSUE 01 — Patroni Failover Not Triggering / Stuck in Leader Election

CRITICAL

Component	Patroni / Percona Operator
Affected Layer	HA Cluster Control Plane
Common Cause	DCS (etcd) unavailability, network partition, clock skew between VMs
Alert Name	patroni_master_is_running == 0 for > 60s
Recovery Time	2–10 min automated; up to 30 min manual

Symptoms

- kubectl get pods shows all Patroni pods running but no leader promoted.
- patronictl list shows all nodes as 'Replica' or blank leader field.
- HAProxy health checks failing on /primary endpoint — all backends marked DOWN.
- Applications receive 'could not connect to the server' errors.
- Operator logs show repeated reconcile errors for PerconaPGCluster.
- Prometheus alert PatroniNoLeader firing for more than 1 minute.

⌨ DIAGNOSTIC COMMANDS

```
# — Patroni cluster state
```

Cluster overview	`kubectl exec -n <ns> <patroni-pod> -- patronictl -c /etc/patroni/patroni.yml list`
Topology view	`kubectl exec -n <ns> <patroni-pod> -- patronictl -c /etc/patroni/patroni.yml topology`

Switchover history	`kubectl exec -n <ns> <patroni-pod> -- patronictl -c /etc/patroni/patroni.yml history`
`# — etcd DCS health ──────────────────────────────────────`	
etcd member health	`kubectl exec -n <ns> <etcd-pod> -- etcdctl endpoint health --cluster`
etcd member list	`kubectl exec -n <ns> <etcd-pod> -- etcdctl member list`
`# — Clock skew check on Alma9 VM nodes ──────────────────`	
NTP sync status	`ssh <node-ip> 'chronyc tracking | grep -E "System time|Stratum"'`
Time diff between nodes	`for n in node1 node2 node3; do echo "$n:"; ssh $n date; done`
`# — Operator and pod logs ──────────────────────────────`	
Operator logs	`kubectl logs -n <ns> deploy/percona-postgresql-operator --tail=200`
Patroni pod logs	`kubectl logs -n <ns> <patroni-pod> -c patroni --tail=200`
Cluster events	`kubectl get events -n <ns> --sort-by=.metadata.creationTimestamp | tail -40`

> ℹ️ **NOTE**
> Clock skew > 500 ms between Kubernetes/VM nodes will cause etcd Raft consensus failures and prevent leader election. Always verify chronyc tracking shows System time offset < 100 ms on all Alma9 nodes before investigating elsewhere.

Resolution Steps

1	Verify etcd cluster health — all 3 or 5 members must be healthy before Patroni can elect a leader.

2	Fix NTP on affected VM nodes: systemctl restart chronyd && chronyc makestep — then re-verify chronyc tracking.
3	If etcd quorum is lost, restore from the most recent snapshot: etcdctl snapshot restore <snap.db> --data-dir=/var/lib/etcd-restore.
4	If Patroni is in split-brain, demote the rogue leader: patronictl -c /etc/patroni/patroni.yml demote <node>.
5	Reinitialize a broken standby: patronictl -c /etc/patroni/patroni.yml reinit <cluster> <node> --force.
6	Trigger controlled switchover to healthy replica: patronictl switchover <cluster> --master <old> --candidate <new> --force.
7	Confirm operator reconciles successfully: kubectl describe perconapgcluster -n <ns> | grep -A5 Status.
8	Validate HAProxy picks up new leader within 10 s: curl http://<haproxy-svc>:8008/primary should return HTTP 200.

Prevention & Alerting

- Prometheus rule: max(patroni_master_is_running) by (cluster) == 0 for 60s → CRITICAL.
- Ensure Chrony is configured on all Alma9 nodes and time offset < 100 ms.
- Use odd etcd member count (3 or 5) to maintain quorum during VM failures.
- Patroni tuning: ttl=30, loop_wait=10, retry_timeout=10 for fast failover.
- Schedule monthly failover drills: patronictl switchover --force in a staging namespace.

ISSUE 02 — PVC / PersistentVolume Mount Failure (CSI Driver) — CRITICAL

Component	Kubernetes Storage / CSI Driver

Affected Layer	Storage / Pod Startup
Common Cause	CSI driver crash, stale VolumeAttachment, disk detach at hypervisor level
Alert Name	KubePersistentVolumeFillingUp, PodNotReady
Recovery Time	15–45 min depending on disk state

Symptoms

- PostgreSQL pod stuck in 'ContainerCreating' or 'Init:0/1' for more than 5 minutes.
- kubectl describe pod shows: 'Unable to attach or mount volumes: timed out waiting for the condition'.
- VolumeAttachment object exists but attachment status shows false.
- CSI node plugin pod logs show dial timeout or gRPC connection errors.
- Disk physically present on Alma9 VM but not visible inside the pod.

DIAGNOSTIC COMMANDS

```
# ── Pod and PVC state ──────────────────────────────────────
```

Describe stuck pod	`kubectl describe pod -n <ns> <pg-pod>`
PVC status	`kubectl get pvc -n <ns>`
PV detail	`kubectl describe pv <pv-name>`

```
# ── VolumeAttachment troubleshooting ──────────────────────
```

List VolumeAttachments	`kubectl get volumeattachment | grep <pv-name>`
Describe attachment	`kubectl describe volumeattachment <va-name>`

```
# ── CSI driver status ──────────────────────────────────────
```

CSI node pods on node	`kubectl get pod -n kube-system -l app=csi-node -o wide | grep <node-name>`
CSI node driver logs	`kubectl logs -n kube-system <csi-node-pod> -c csi-driver --tail=100`

```
# ── Disk check on Alma9 VM ──────────────────────────────────
```

Block devices on VM	`ssh <node-ip> 'lsblk'`
Kernel disk messages	`ssh <node-ip> 'dmesg | tail -30'`
Mount table	`ssh <node-ip> 'df -h && mount | grep /dev/sd'`

Resolution Steps

1	SSH into the Alma9 VM node and run lsblk to confirm whether the disk is visible at the OS level.
2	If disk is visible but not mounted inside pod: run 'udevadm trigger' on the node to re-detect the device.
3	Delete the stale VolumeAttachment: kubectl delete volumeattachment <va-name> — Kubernetes will recreate it and reattempt attachment.
4	If the CSI node plugin is crashed, restart the DaemonSet: kubectl rollout restart daemonset/<csi-node-ds> -n kube-system.
5	If the PVC is bound to a dead node, cordon the node then delete the pod to reschedule: kubectl cordon <node> && kubectl delete pod <pg-pod> -n <ns>.
6	For persistent detach failures, force-unmount at OS level on the old node: umount -f /var/lib/kubelet/pods/<uid>/volumes/... then delete the VolumeAttachment.
7	After recovery, verify data integrity: kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c 'SELECT pg_postmaster_start_time();'

Prevention

- Pin CSI driver to a tested release; configure automatic restart policy on CSI pods.
- Prometheus alert: kube_persistentvolumeclaim_status_phase{phase='Lost'} == 1 → CRITICAL.
- Use PodDisruptionBudgets to prevent simultaneous rescheduling of multiple Postgres pods.
- Set StorageClass reclaimPolicy: Retain to prevent accidental data loss on PVC deletion.

ISSUE 03 — PGBouncer Connection Pool Exhaustion — HIGH

Component	PGBouncer
Affected Layer	Connection Management
Common Cause	Long-running/idle transactions, pool_size misconfiguration, connection leaks in app code
Alert Name	pgbouncer_pool_waiting_clients > 50
Recovery Time	5–15 min

Symptoms

- Applications receive 'FATAL: remaining connection slots are reserved' or connection timeout errors.
- PGBouncer SHOW POOLS shows cl_waiting > 0 and maxwait continuously increasing.
- pg_stat_activity on the primary shows connection count near max_connections.
- New application instances cannot connect while existing sessions are fine.
- Prometheus: pgbouncer_pool_waiting_clients_total rising and not recovering.

⌨ DIAGNOSTIC COMMANDS

```
# — PGBouncer admin console (connect via kubectl exec)
```

Connect to admin	`kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U pgbouncer pgbouncer`
Pool status	`kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U pgbouncer pgbouncer -c 'SHOW POOLS;'`
Client list	`kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U pgbouncer pgbouncer -c 'SHOW CLIENTS;'`
Server connections	`kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U pgbouncer pgbouncer -c 'SHOW SERVERS;'`
Stats per DB	`kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U pgbouncer pgbouncer -c 'SHOW STATS;'`

```
# ── PostgreSQL connection analysis ──────────────────────────
```

Connection count by state	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT count(*), state FROM pg_stat_activity GROUP BY state;"`
Idle-in-transaction sessions	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT pid, now()-xact_start AS duration, application_name, query \ FROM pg_stat_activity WHERE state = 'idle in transaction' \ ORDER BY duration DESC LIMIT 20;"`
Oldest transaction age	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT max(now()-xact_start) FROM pg_stat_activity WHERE state != 'idle';"`

> ⚠ **WARNING**
> Do NOT blindly terminate all connections. Identify the root cause first. Killing legitimate transactions can cause data inconsistency or application errors.

Resolution Steps

1	Run SHOW POOLS in PGBouncer admin — identify which database/user pair has the most cl_waiting clients.
2	Kill idle-in-transaction sessions older than 5 minutes: SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE state='idle in transaction' AND now()-xact_start > interval '5 minutes';
3	Pause the saturated database in PGBouncer to queue new connections

	gracefully: PAUSE <dbname>; in admin console.
4	Temporarily increase server_pool_size in the PGBouncer ConfigMap and hot-reload: RELOAD; in PGBouncer admin.
5	Resume the database: RESUME <dbname>; — verify cl_waiting drops to 0.
6	Set idle_transaction_timeout = 60 in pgbouncer.ini to auto-terminate future stale transactions.
7	Investigate application code for connection leak patterns — look for missing connection.close() calls.

Recommended PGBouncer Configuration

# [pgbouncer] section — transaction pooling for OLTP workloads	
Pooling mode	pool_mode = transaction
Max client connections	max_client_conn = 5000
Pool size per pair	default_pool_size = 25
Reserve pool	reserve_pool_size = 10 ; fallback for burst
Hard DB cap	max_db_connections = 100 ; total to PostgreSQL
Kill idle transactions	idle_transaction_timeout = 60
Auth method	auth_type = scram-sha-256

ISSUE 04 — HAProxy Backend Health Check Failures — HIGH

Component	HAProxy
Affected Layer	Load Balancer / Traffic Routing

Common Cause	Patroni REST API down, NetworkPolicy blocking port 8008, stale pod IPs in HAProxy config
Alert Name	haproxy_backend_active_servers == 0
Recovery Time	5–20 min

Symptoms

- All application connections are refused or routed to wrong backend.
- HAProxy stats page shows all backends in DOWN or DRAIN state.
- Patroni REST health endpoint (port 8008) not responding from the HAProxy pod.
- kubectl get endpoints shows empty endpoint list for HAProxy service.
- Prometheus alert HAProxyBackendDown firing.

DIAGNOSTIC COMMANDS

— HAProxy stats and config

Backend stats via socket	`kubectl exec -n <ns> <haproxy-pod> -- sh -c \ 'echo "show stat" | socat stdio /var/run/haproxy/admin.sock | cut -d, -f1,2,18,19,20'`
Validate HAProxy config	`kubectl exec -n <ns> <haproxy-pod> -- haproxy -c -f /etc/haproxy/haproxy.cfg`
HAProxy logs (last 100)	`kubectl logs -n <ns> <haproxy-pod> --tail=100`

— Test Patroni health endpoints from HAProxy pod

Test primary endpoint	`kubectl exec -n <ns> <haproxy-pod> -- curl -sv http://<patroni-pod-ip>:8008/primary`
Test replica endpoint	`kubectl exec -n <ns> <haproxy-pod> -- curl -sv http://<patroni-pod-ip>:8008/replica`
Test /health endpoint	`kubectl exec -n <ns> <haproxy-pod> -- curl -sv http://<patroni-pod-ip>:8008/health`

— Network policy and service

List network policies	`kubectl get networkpolicy -n <ns>`
Check service endpoints	`kubectl get endpoints -n <ns> <pg-service>`
Check pod labels	`kubectl get pod -n <ns> --show-labels | grep postgres`

Resolution Steps

1	Access HAProxy stats (port 1936) or admin socket to identify which specific backends are DOWN.
2	From the HAProxy pod, curl the Patroni /primary endpoint on port 8008 for each PostgreSQL pod IP. HTTP 200 = healthy; HTTP 503 = not primary.
3	If Patroni endpoints are unreachable, check NetworkPolicy: must allow TCP ingress on port 8008 from HAProxy pods to PostgreSQL pods.
4	If HAProxy config has stale pod IPs (not using DNS service names), update the ConfigMap backend section to use Kubernetes service DNS names instead of IPs.
5	Reload HAProxy config without downtime: kubectl exec -n <ns> <haproxy-pod> -- haproxy -sf $(cat /var/run/haproxy.pid) -f /etc/haproxy/haproxy.cfg.
6	If the HAProxy pod itself is crashed, delete it and let the Deployment restart it: kubectl delete pod <haproxy-pod> -n <ns>.
7	Verify recovery: all primary backends should show UP in stats and curl /primary should return 200.

ISSUE 05 — High Replication Lag on Standby HIGH

Component	PostgreSQL Streaming Replication / Patroni
Affected Layer	Data Replication
Common Cause	Network saturation, disk I/O bottleneck, long autovacuum, WAL receiver crash, inactive slots
Alert Name	pg_replication_lag > 30s, pg_replication_slot_inactive
Recovery Time	10–60 min depending on cause

Symptoms

- Prometheus alert: PostgreSQLReplicationLagCritical — pg_replication_lag_seconds > 30.
- patronictl list shows replica lag value continuously increasing.
- Read queries on replica returning stale or outdated data.
- WAL receiver process absent from pg_stat_replication on the primary.
- In extreme cases, Patroni demotes the replica and marks it as 'stopped'.

⌨ DIAGNOSTIC COMMANDS

```
# — On primary: replication state ————————————————————
```

Replication status

```
kubectl exec -n <ns> <pg-primary> -- psql -U postgres -c \   "SELECT client_addr, state, sent_lsn, replay_lsn,    pg_size_pretty(sent_lsn - replay_lsn) AS lag FROM pg_stat_replication;"
```

Replication slot lag

```
kubectl exec -n <ns> <pg-primary> -- psql -U postgres -c \   "SELECT slot_name, active, pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),restart_lsn)) AS lag FROM pg_replication_slots;"
```

```
# — On replica: WAL receiver ————————————————————————
```

WAL receiver status

```
kubectl exec -n <ns> <pg-replica> -- psql -U postgres -c \   "SELECT status, received_lsn, last_msg_receipt_time FROM pg_stat_wal_receiver;"
```

```
# — Node-level I/O and network ——————————————————————
```

Disk I/O on

```
ssh <node-ip> 'iostat -x 2 5'
```

node	
Network stats	`ssh <node-ip> 'sar -n DEV 2 5'`
Autovacuum check	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT pid, now()-query_start AS age, query FROM pg_stat_activity WHERE query LIKE '%autovacuum%';"`

Resolution Steps

1	Check pg_stat_wal_receiver on the replica. If empty, the WAL receiver crashed — Patroni should restart it automatically; if not, delete and restart the replica pod.
2	If disk I/O is the bottleneck (iostat shows >80% utilisation), identify which autovacuum or query is causing I/O and cancel it if safe.
3	Check for inactive replication slots hoarding WAL: drop slots that are no longer active — SELECT pg_drop_replication_slot('slot_name');
4	Enable WAL compression on the primary to reduce network load: ALTER SYSTEM SET wal_compression = on; SELECT pg_reload_conf();
5	If the replica has fallen so far behind that WAL is gone, Patroni will auto-reinitialise it from a base backup: patronictl reinit <cluster> <node>.
6	After recovery, verify lag returns to < 1 s in pg_stat_replication and Patroni reports all replicas as in-sync.

Prevention

- Set wal_keep_size = 2048 MB or use replication slots with lag monitoring.
- Alert on inactive replication slots: pg_replication_slots where active=false and lag > 1 GB → CRITICAL.
- Configure max_slot_wal_keep_size = 20 GB to cap disk usage from stale slots.
- Dedicate a separate network interface or VLAN for replication traffic on the Alma9 VMs.

ISSUE 06 — PostgreSQL Pod OOMKilled / Memory Pressure — CRITICAL

Component	PostgreSQL Pod / Kubernetes Node
Affected Layer	Resource Management
Common Cause	shared_buffers too large, work_mem over-allocation, missing pod memory limits
Alert Name	container_memory_working_set near limit, OOMKilled exit code 137
Recovery Time	5–20 min to restore + tune

Symptoms

- kubectl get pods shows a PostgreSQL pod with status OOMKilled.
- kubectl describe pod shows: 'Last State: OOMKilled, Exit Code: 137'.
- Node memory pressure may trigger eviction of co-located pods on the same VM.
- Sudden connection drops preceding the OOM event visible in application logs.
- Prometheus: container_memory_working_set_bytes > 90% of limit for more than 5 minutes.

⌨ DIAGNOSTIC COMMANDS

```
# — OOM event investigation ————————————————————————
```

OOM state in pod

```
kubectl describe pod -n <ns> <pg-pod> | grep -A10 'Last State'
```

OOM events in namespace

```
kubectl get events -n <ns> | grep OOMKill
```

Kernel OOM log on VM

```
ssh <node-ip> 'dmesg | grep -i oom | tail -20'
```

```
# — Memory configuration and usage ——————————————————
```

PostgreSQL memory

```
kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \
  "SELECT name, setting, unit FROM pg_settings
WHERE name IN \
```

settings	`('shared_buffers','work_mem','maintenance_work_mem','max_connections');"`
Pod resource limits	`kubectl get pod -n <ns> <pg-pod> -o jsonpath='{.spec.containers[*].resources}'`
Live pod memory usage	`kubectl top pod -n <ns>`
Parallel query workers	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT count(*) FROM pg_stat_activity WHERE state='active' AND wait_event_type='Lock';"`

ℹ️ NOTE
Each parallel query worker can consume up to work_mem independently. With 200 connections and max_parallel_workers_per_gather=4, worst-case memory = 200 × 4 × work_mem. Always calculate peak memory usage before setting work_mem.

Resolution Steps

1	Confirm OOM kill via dmesg and kubectl events — identify which process the kernel killed.
2	Verify Patroni performed automatic failover. If so, confirm the new leader is healthy before proceeding.
3	Calculate safe work_mem: (pod_memory_limit - shared_buffers) / (max_connections × 3). For 32 GB pod with 8 GB shared_buffers: (24 GB / 600) ≈ 40 MB.
4	Apply new work_mem via ALTER SYSTEM: ALTER SYSTEM SET work_mem = '40MB'; SELECT pg_reload_conf();
5	If the workload genuinely requires more memory, update the PerconaPGCluster CR to increase pod memory limits.
6	Enable Linux huge pages on Alma9 VMs to reduce memory management overhead: echo 'vm.nr_hugepages=512' >> /etc/sysctl.conf && sysctl -p.
7	Set Kubernetes memory requests == limits for PostgreSQL pods to get Guaranteed QoS class and avoid OOM eviction.

ISSUE 07 — Disk Space Exhaustion on PostgreSQL Persistent Volume

CRITICAL

Component	PostgreSQL Storage / PVC
Affected Layer	Storage
Common Cause	Table/index bloat, WAL accumulation, inactive replication slots, unrotated log files
Alert Name	KubePersistentVolumeFillingUp > 85%, kubelet_volume_stats_used_bytes high
Recovery Time	15–60 min for space recovery; hours for PVC resize

Symptoms

- PostgreSQL goes read-only or stops with 'No space left on device' error.
- Patroni marks the node as unhealthy due to inability to write WAL segments.
- Prometheus: KubePersistentVolumeFillingUp alert at 85% and 95% thresholds.
- kubectl exec shows df -h reporting 100% usage on /pgdata mount.
- pg_database_size() returning unexpectedly large values.

⌨ DIAGNOSTIC COMMANDS

```
# — Disk usage inside pod
```

Overall disk usage

```
kubectl exec -n <ns> <pg-pod> -- df -h
```

Top directories by size

```
kubectl exec -n <ns> <pg-pod> -- du -sh /pgdata/* | sort -rh | head -20
```

WAL directory size

```
kubectl exec -n <ns> <pg-pod> -- du -sh /pgdata/pg_wal/
```

```
# — Database and table sizes
```

————————————————————————	
Database sizes	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT datname, pg_size_pretty(pg_database_size(datname)) FROM pg_database ORDER BY 2 DESC;"`
Top bloated tables	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -d <db> -c \ "SELECT schemaname, tablename, n_dead_tup, pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS total FROM pg_stat_user_tables ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC LIMIT 20;"`
# — Replication slot WAL retention ————————————————————	
Slot WAL lag	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT slot_name, active, pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),restart_lsn)) AS lag FROM pg_replication_slots;"`

Immediate Space Recovery

1	Drop inactive replication slots that are consuming WAL: SELECT pg_drop_replication_slot('slot_name') WHERE slot_name IN (SELECT slot_name FROM pg_replication_slots WHERE active = false);
2	Delete old PostgreSQL log files: find /pgdata/log -name '*.log' -mtime +7 -delete.
3	Run VACUUM ANALYZE on the most bloated tables to reclaim dead tuple space.
4	For severe bloat without downtime, use pg_repack extension: pg_repack -t <tablename> -d <dbname>.
5	If the CSI driver supports PVC expansion, patch the PVC: kubectl patch pvc <pvc-name> -n <ns> -p '{"spec":{"resources":{"requests":{"storage":"500Gi"}}}}'.
6	Monitor df -h inside the pod every 2 minutes until usage drops below 75%.
7	After recovery, investigate root cause (bloat, log rotation policy, or replication slot leak) before marking resolved.

Prevention

- Set autovacuum_vacuum_scale_factor = 0.01 and autovacuum_analyze_scale_factor = 0.005 for large tables.
- Alert at 75% (WARN), 85% (HIGH), 95% (CRITICAL) PVC usage.
- Set max_slot_wal_keep_size = 20 GB to cap WAL retained by replication slots.
- Configure log rotation: logging_collector = on, log_rotation_age = 1d, log_rotation_size = 500MB.
- Implement pg_partman for time-based table partitioning to DROP old partitions instead of running VACUUM.

ISSUE 08 — Percona Operator CRD Reconciliation Errors | HIGH

Component	Percona PostgreSQL Operator
Affected Layer	Kubernetes Control Plane
Common Cause	CRD version mismatch after upgrade, missing RBAC permissions, webhook TLS failure
Alert Name	Operator pod CrashLoopBackOff, PerconaPGCluster status degraded
Recovery Time	15–45 min

Symptoms

- Percona operator pod in CrashLoopBackOff or Error state.
- PerconaPGCluster CR status shows 'error' or fails to transition to 'ready'.
- Changes to PerconaPGCluster CR are not being applied — cluster ignores spec updates.
- kubectl describe perconapgcluster shows 'reconcile failed' in events.
- New pods not starting when cluster is scaled up via CR modification.

DIAGNOSTIC COMMANDS

```
# ── Operator health ──────────────────────────────────────────────
```

Operator pod status	`kubectl get pod -n <ns> -l app.kubernetes.io/name=percona-postgresql-operator`
Operator logs	`kubectl logs -n <ns> deploy/percona-postgresql-operator --tail=200`

```
# ── CR and CRD state ─────────────────────────────────────────────
```

Cluster CR status	`kubectl get perconapgcluster -n <ns> -o wide`
Cluster CR detail	`kubectl describe perconapgcluster -n <ns> <cluster-name>`
CRD installed version	`kubectl get crd perconapgclusters.pgv2.percona.com -o jsonpath='{.spec.versions[*].name}'`

```
# ── RBAC and webhook checks ──────────────────────────────────────
```

Operator ClusterRole	`kubectl describe clusterrole percona-postgresql-operator`
RBAC bindings	`kubectl get clusterrolebinding | grep percona`
Webhook config	`kubectl describe validatingwebhookconfiguration percona-postgresql-operator-webhook`

Resolution Steps

1	Read operator logs carefully — the error is usually explicit: 'cannot list resource', 'admission webhook denied', or 'unknown field in spec'.
2	For RBAC issues: re-apply operator RBAC manifests: helm upgrade percona-operator percona/pg-operator -n <ns>.
3	For CRD schema validation errors after upgrade: check operator changelog for renamed or deprecated CR fields and update the PerconaPGCluster CR spec accordingly.

4	For webhook TLS failures: delete the webhook cert secret so cert-manager or the operator recreates it: kubectl delete secret percona-postgresql-operator-webhook-cert -n <ns>.
5	If the operator is crash-looping due to a panic: collect full logs, then force restart: kubectl rollout restart deploy/percona-postgresql-operator -n <ns>.
6	Force a reconcile by annotating the CR: kubectl annotate perconapgcluster <name> -n <ns> reconcile=$(date +%s) --overwrite.
7	Confirm CR status transitions to 'ready' and all StatefulSet pods are Running.

ISSUE 09 — Slow Queries / Lock Contention — MEDIUM

Component	PostgreSQL Query Engine
Affected Layer	Database Performance
Common Cause	Missing indexes, table bloat, lock conflicts, bad query plans, autovacuum interference
Alert Name	pg_stat_statements slow queries, lock_wait exceeded
Recovery Time	Minutes for lock kills; hours for optimisation

Symptoms

- Application latency spikes; specific queries consistently taking >1 s.
- pg_stat_activity shows many sessions in 'lock wait' state.
- pg_locks shows conflicting lock requests forming a blocking chain.
- Prometheus: PostgreSQLTooManyLocksAcquired or slow_queries > threshold.
- autovacuum holding AccessShareLock blocking urgent DDL operations.

⌨ DIAGNOSTIC COMMANDS

# — Lock and blocking analysis	
Blocking query chain	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT blocked.pid, blocked.query AS blocked_query, blocker.pid AS blocker_pid, blocker.query AS blocker_query, now()-blocked.query_start AS wait_duration FROM pg_stat_activity blocked JOIN pg_stat_activity blocker ON blocker.pid = ANY(pg_blocking_pids(blocked.pid)) ORDER BY wait_duration DESC;"`
All active locks	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT relation::regclass, mode, granted, pid FROM pg_locks WHERE relation IS NOT NULL ORDER BY granted;"`
# — Slow query analysis via pg_stat_statements	
Top slow queries	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -d <db> -c \ "SELECT left(query,80), calls, round(mean_exec_time::numeric,2) AS avg_ms, total_exec_time FROM pg_stat_statements ORDER BY mean_exec_time DESC LIMIT 20;"`
Tables missing indexes	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -d <db> -c \ "SELECT schemaname, tablename, seq_scan, idx_scan FROM pg_stat_user_tables WHERE seq_scan > 500 ORDER BY seq_scan-idx_scan DESC LIMIT 20;"`

Resolution Steps

1	Identify the root blocker via the blocking query chain query; terminate it only if safe: SELECT pg_terminate_backend(<blocker_pid>);
2	Run EXPLAIN (ANALYZE, BUFFERS) on the slowest queries to find Seq Scans and high-cost nodes.
3	Create missing indexes concurrently to avoid locking production: CREATE INDEX CONCURRENTLY idx_name ON table (column);
4	If autovacuum is blocking critical DDL: cancel it with pg_cancel_backend(pid) — it will restart on the next autovacuum cycle.
5	Set lock_timeout = '30s' at database level to prevent indefinite lock waits:

	ALTER DATABASE <db> SET lock_timeout = '30s';
6	Review and tune: checkpoint_completion_target = 0.9, random_page_cost = 1.1 (SSD-backed CSI volumes), effective_io_concurrency = 200.

ISSUE 10 — WAL Archiving Failure / pgBackRest Errors — HIGH

Component	WAL Archiving / pgBackRest
Affected Layer	Backup & Recovery
Common Cause	S3/NFS target unreachable, expired credentials, archive target full, permission errors
Alert Name	pg_archiver_failed, pgbackrest_last_backup_age > 24 h
Recovery Time	15–60 min

Symptoms

- pg_stat_archiver shows last_failed_time recent and last_archived_time stale.
- PostgreSQL logs repeatedly show: 'archiving of segment failed; will retry'.
- WAL files accumulating in /pgdata/pg_wal (disk space filling rapidly).
- pgBackRest backup CRs showing 'Failed' status in Percona operator.
- Prometheus alert: pgbackrest_last_backup_age firing.

⌨ DIAGNOSTIC COMMANDS

```
# — Archiver and pgBackRest status
```

Archiver status

```
kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \   "SELECT archived_count, last_archived_wal, last_failed_wal, last_failed_time FROM pg_stat_archiver;"
```

pgBackRest info

```
kubectl exec -n <ns> <pg-pod> -- pgbackrest info
```

pgBackRest check	`kubectl exec -n <ns> <pg-pod> -- pgbackrest check --stanza=<stanza>`
pgBackRest logs	`kubectl exec -n <ns> <pg-pod> -- tail -100 /var/log/pgbackrest/pgbackrest.log`

```
# ── Operator backup status ──────────────────────────────────────
```

List backup CRs	`kubectl get perconapgbackup -n <ns>`
Describe failed backup	`kubectl describe perconapgbackup -n <ns> <backup-name>`

```
# ── Test archive push manually ──────────────────────────────────
```

Force archive push test	`kubectl exec -n <ns> <pg-pod> -- pgbackrest archive-push \ --stanza=<stanza> --log-level-console=info \ /pgdata/pg_wal/000000010000000000000001`

Resolution Steps

1	Check pg_stat_archiver.last_failed_wal to identify which WAL segment is failing and the error message.
2	Verify the archive target (S3 bucket or NFS mount) is accessible from the PostgreSQL pod — run a connectivity test.
3	If using S3: confirm credentials in the pgBackRest Kubernetes Secret are valid and not expired. Update the Secret and restart the pod if needed.
4	If using NFS: check mount status inside pod (mount | grep nfs) and on the VM node (showmount -e <nfs-server>).
5	Re-push the failed WAL segment manually to validate the fix: pgbackrest archive-push --stanza=<stanza> <wal-path>.
6	Run a full pgBackRest check: pgbackrest check --stanza=<stanza> — expect no errors.
7	If the archive backlog is very large, trigger a new full backup: kubectl apply -f full-backup-cr.yaml.

8	Verify pg_stat_archiver.archived_count increments every few seconds and last_failed_time is no longer updating.

ISSUE 11 — Certificate Expiry (TLS for Patroni / PGBouncer) HIGH

Component	TLS / cert-manager / Patroni / PGBouncer
Affected Layer	Security / Connectivity
Common Cause	cert-manager renewal failure, manual certificates not rotated, clock drift
Alert Name	ssl_certificate_expiry_days < 30, TLS handshake failure
Recovery Time	10–30 min

Symptoms

- Applications fail with 'SSL SYSCALL error: EOF detected' or 'certificate has expired'.
- Patroni cluster members fail to communicate over TLS — split-brain risk.
- PGBouncer refuses connections with 'SSL handshake error' in logs.
- kubectl describe certificate shows 'Not Ready' or past expiry date.
- Prometheus blackbox exporter alert: ssl_expiry_days < 30.

DIAGNOSTIC COMMANDS

```
# — Certificate expiry checks
```

Cert expiry inside pod	`kubectl exec -n <ns> <pg-pod> -- openssl x509 -in /etc/ssl/certs/server.crt -noout -dates`
Cert from Kubernetes secret	`kubectl get secret -n <ns> <tls-secret> -o jsonpath='{.data.tls\.crt}' | \ base64 -d | openssl x509 -noout -dates`
Live TLS	`openssl s_client -connect <pg-svc>:5432 -starttls postgres 2>&1 | grep -E 'subject|Verify|expire'`

handshake test	
# — cert-manager state	
cert-manager Certificate resources	`kubectl get certificate -n <ns>`
Certificate detail	`kubectl describe certificate -n <ns> <cert-name>`
CertificateRequest status	`kubectl get certificaterequest -n <ns>`
cert-manager logs	`kubectl logs -n cert-manager deploy/cert-manager --tail=100 | grep -i error`

Resolution Steps

1	Confirm expiry: openssl x509 -in <cert> -noout -enddate — check the exact expiry date.
2	If using cert-manager: force immediate renewal by annotating the Certificate: kubectl annotate certificate <cert-name> -n <ns> cert-manager.io/issuer-name=<issuer> --overwrite.
3	Alternatively, delete the CertificateRequest to trigger immediate re-issuance: kubectl delete certificaterequest -n <ns> <cr-name>.
4	If using manual/self-signed certificates: generate new certs with the same CA and update the Secret: kubectl create secret tls <secret> --cert=new.crt --key=new.key --dry-run=client -o yaml | kubectl apply -f -.
5	Trigger a rolling restart of Patroni pods to load new certs (done via operator annotation): kubectl annotate perconapgcluster <name> -n <ns> kubectl.kubernetes.io/restartedAt=$(date -u +%s) --overwrite.
6	Verify connectivity post-rotation: openssl s_client -connect <pg-svc>:5432 -starttls postgres should show 'Verify return code: 0 (ok)'.

ISSUE 12 — Kubernetes Node NotReady (VM Failure / Network Partition) — CRITICAL

Component	Kubernetes Node / Alma9 VM
Affected Layer	Infrastructure
Common Cause	VM crash, kernel panic, OOM at node level, NIC failure, storage I/O hang
Alert Name	KubeNodeNotReady, node_exporter metrics absent
Recovery Time	10–60 min depending on hardware state

Symptoms

- kubectl get nodes shows a node in 'NotReady' state.
- Postgres pods on the affected node show 'Unknown' status and are not rescheduled.
- Patroni may not trigger failover immediately if the leader pod is stuck in 'Unknown'.
- Prometheus: KubeNodeNotReady alert firing; node_exporter metrics stop arriving.
- No SSH access to the VM; hypervisor console shows panic or hung state.

⌨ DIAGNOSTIC COMMANDS

```
# — Node status investigation ————————————————————
```

Node conditions	`kubectl describe node <node-name> | grep -A25 Conditions`
Pods on affected node	`kubectl get pod --all-namespaces --field-selector spec.nodeName=<node-name>`
Node memory/CPU pressure	`kubectl get node <node-name> -o jsonpath='{.status.conditions}' | python3 -m json.tool`

```
# — Kubelet on Alma9 VM (if SSH accessible) ——————————
```

kubelet service status	`ssh <node-ip> 'systemctl status kubelet'`
kubelet logs	`ssh <node-ip> 'journalctl -u kubelet --since "10 minutes ago" | tail -100'`
System memory / OOM	`ssh <node-ip> 'dmesg | grep -Ei "oom|panic|error" | tail -30'`
Disk health	`ssh <node-ip> 'smartctl -a /dev/sda | grep -E "Health|Error|Temperature"'`

Resolution Steps

1	Attempt SSH to the Alma9 VM. If unreachable, use the hypervisor console (IPMI/iDRAC/iLO or private cloud console).
2	If the VM has a kernel panic or OOM kill loop: hard-reboot the VM via the hypervisor API or management console.
3	After VM restart, verify kubelet recovers: systemctl status kubelet && systemctl enable kubelet.
4	If the node remains NotReady after VM recovery, drain and remove it: kubectl drain <node> --ignore-daemonsets --delete-emptydir-data --force.
5	Force-delete pods stuck in Unknown/Terminating (only after confirming node is truly offline): kubectl delete pod <pod> -n <ns> --grace-period=0 --force.
6	Patroni will trigger failover once the leader pod is definitively removed. Monitor: patronictl list until a new leader is elected.
7	Uncordon the node after full recovery: kubectl uncordon <node> — allow pods to reschedule gradually.
8	Conduct root cause analysis: review hypervisor logs, disk SMART data, and kernel OOM messages.

ISSUE 13 — autovacuum Not Running / Table Bloat Buildup — MEDIUM

Component	PostgreSQL autovacuum
Affected Layer	Database Maintenance
Common Cause	Long-running transactions blocking vacuum horizon, autovacuum misconfigured, too few workers
Alert Name	pg_stat_user_tables n_dead_tup > 5 M, bloat ratio > 50%
Recovery Time	Minutes to fix; hours to fully reclaim space

Symptoms

- Table sizes growing despite low data growth; pg_total_relation_size >> pg_relation_size.
- Query performance degrading on specific tables due to dead tuple overhead.
- n_dead_tup in the millions for high-traffic tables in pg_stat_user_tables.
- Transaction ID wraparound warning appearing in PostgreSQL logs.
- Prometheus alert: dead tuple ratio (n_dead_tup / n_live_tup) > 0.2 for key tables.

⌨ DIAGNOSTIC COMMANDS

```
# — Table bloat and vacuum status
```

Bloat by table

```
kubectl exec -n <ns> <pg-pod> -- psql -U postgres -d <db> -c \   "SELECT schemaname, tablename, n_dead_tup, n_live_tup, round(n_dead_tup::numeric/(n_live_tup+n_dead_tup+1)*100,2) AS dead_pct,    last_autovacuum FROM pg_stat_user_tables    WHERE n_dead_tup > 100000 ORDER BY dead_pct DESC LIMIT 20;"
```

Transaction ID wraparound risk

```
kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \   "SELECT datname, age(datfrozenxid) AS xid_age FROM pg_database ORDER BY xid_age DESC;"
```

Autovacuum workers running

```
kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \   "SELECT pid, now()-query_start AS age, query FROM pg_stat_activity WHERE query LIKE 'autovacuum%';"
```

```
# — Check what is blocking vacuum
```

Oldest transaction (blocks vacuum)	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT pid, now()-xact_start AS age, state, left(query,80) FROM pg_stat_activity WHERE state != 'idle' ORDER BY xact_start LIMIT 5;"`

Resolution Steps

1	Terminate any long-running idle-in-transaction sessions blocking the vacuum horizon.
2	Manually vacuum the most bloated tables: VACUUM (VERBOSE, ANALYZE) schema.tablename;
3	For severe bloat, run pg_repack for online bloat removal (no table lock required): pg_repack -t schema.table -d dbname.
4	Increase autovacuum workers: ALTER SYSTEM SET autovacuum_max_workers = 6; SELECT pg_reload_conf();
5	Tune per-table autovacuum for high-write tables: ALTER TABLE orders SET (autovacuum_vacuum_scale_factor = 0.01, autovacuum_vacuum_cost_limit = 800);
6	If transaction ID age > 1.5 billion: reduce vacuum_freeze_min_age temporarily and run VACUUM FREEZE on affected tables immediately to prevent wraparound shutdown.

ISSUE 14 — PGBouncer Authentication Failures — MEDIUM

Component	PGBouncer
Affected Layer	Authentication
Common Cause	Stale userlist.txt, md5/scram-sha-256 mismatch, pg_hba.conf blocking PGBouncer IP
Alert Name	pgbouncer_login_failed_count increasing, application

	FATAL auth errors
Recovery Time	10–20 min

Symptoms

- Applications receive 'FATAL: password authentication failed for user' from PGBouncer.
- PGBouncer logs show 'no such user' or 'auth_query failed'.
- After a PostgreSQL password rotation, all new connections begin failing.
- Existing long-lived PGBouncer connections continue to work while new ones fail.
- SHOW CLIENTS in PGBouncer admin shows clients stuck in 'login' state.

⌨ DIAGNOSTIC COMMANDS

# — PGBouncer authentication investigation	
PGBouncer auth error logs	`kubectl logs -n <ns> <pgb-pod> --tail=100 | grep -iE 'auth|password|fatal|FATAL'`
userlist.txt content	`kubectl exec -n <ns> <pgb-pod> -- cat /etc/pgbouncer/userlist.txt`
PGBouncer auth settings	`kubectl exec -n <ns> <pgb-pod> -- grep -i auth /etc/pgbouncer/pgbouncer.ini`
# — Verify PostgreSQL side	
Check pg_shadow for user	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT usename, passwd FROM pg_shadow WHERE usename='<appuser>';"`
pg_hba.conf content	`kubectl exec -n <ns> <pg-pod> -- cat /pgdata/pg_hba.conf | grep -v '^#'`
Verify auth_query works	`kubectl exec -n <ns> <pg-pod> -- psql -U pgbouncer postgres -c \ "SELECT usename, passwd FROM pg_shadow WHERE usename='<appuser>';"`

Resolution Steps

1	Identify whether the failure is: wrong password, user missing from userlist.txt, or pg_hba.conf denying the PGBouncer pod IP.
2	If using auth_query mode, verify the pgbouncer database user has SELECT on pg_shadow: GRANT pg_shadow TO pgbouncer; in PostgreSQL.
3	After password rotation: update the userlist.txt ConfigMap with the new scram-sha-256 hash and hot-reload PGBouncer: RELOAD; in admin console.
4	For scram-sha-256: ensure both pg_hba.conf (scram-sha-256 method) and pgbouncer.ini (auth_type = scram-sha-256) are aligned.
5	Add or fix the pg_hba.conf entry for the PGBouncer pod CIDR: host <db> <user> <pgb-pod-cidr>/32 scram-sha-256.
6	Reload PostgreSQL pg_hba.conf: SELECT pg_reload_conf(); — no restart needed.
7	Test end-to-end: psql -h <pgbouncer-svc> -U <user> -d <db> should succeed.

ISSUE 15 — Prometheus Metrics Not Scraping PostgreSQL Exporter

MEDIUM

Component	postgres_exporter / Prometheus
Affected Layer	Monitoring
Common Cause	ServiceMonitor misconfigured, NetworkPolicy blocking scrape port, wrong credentials, exporter crash
Alert Name	up{job='postgres-exporter'} == 0, target missing from Prometheus
Recovery Time	10–30 min

Symptoms

- Prometheus targets page shows postgres-exporter as DOWN or the target is missing entirely.
- Grafana PostgreSQL dashboards showing 'No data' for all panels.
- Prometheus alert: PostgresExporterDown firing.
- postgres_exporter pod logs showing connection refused or authentication error to PostgreSQL.
- Missing pg_up, pg_stat_activity, and pg_replication metrics.

⌨ DIAGNOSTIC COMMANDS

# — Exporter pod health	
Exporter pod status	`kubectl get pod -n <ns> -l app=postgres-exporter`
Exporter logs	`kubectl logs -n <ns> <exporter-pod> --tail=100`
Test metrics endpoint	`kubectl exec -n <ns> <exporter-pod> -- curl -s http://localhost:9187/metrics | head -30`
# — Prometheus ServiceMonitor	
List ServiceMonitors	`kubectl get servicemonitor -n <ns>`
ServiceMonitor detail	`kubectl describe servicemonitor -n <ns> <sm-name>`
# — Credentials and network	
DSN secret value	`kubectl get secret -n <ns> postgres-exporter-secret \ -o jsonpath='{.data.DATA_SOURCE_NAME}' | base64 -d`
Network policy list	`kubectl get networkpolicy -n <ns>`
Exporter user grants	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT has_function_privilege('postgres_exporter',`

```
                    'pg_stat_statements', 'execute');"
```

Resolution Steps

1	Verify the postgres_exporter pod is Running and its logs show a successful connection to PostgreSQL.
2	Test the metrics endpoint directly from inside the pod: curl http://localhost:9187/metrics. If this fails, fix the exporter first.
3	Check DATA_SOURCE_NAME in the Secret — ensure the DSN points to the correct PostgreSQL service and credentials are valid.
4	Ensure the exporter user has required system grants: GRANT pg_monitor TO postgres_exporter; in PostgreSQL.
5	Check that the ServiceMonitor selector labels match the exporter Service labels — a label mismatch means Prometheus never discovers the target.
6	Check NetworkPolicy: Prometheus pods must be allowed to reach the exporter pod on port 9187.
7	If using Percona PMM integration, verify the pmm-client container in the PostgreSQL pod is healthy: kubectl exec -n <ns> <pg-pod> -c pmm-client -- pmm-admin status.
8	Restart the exporter to pick up any config changes: kubectl rollout restart deployment/postgres-exporter -n <ns>.

ISSUE 16 — Pod CrashLoopBackOff on PostgreSQL Container

CRITICAL

Component	PostgreSQL Pod
Affected Layer	Application Runtime
Common Cause	Data directory corruption, config syntax error, init container failure, wrong filesystem permissions

Alert Name	kube_pod_container_status_restarts_total increasing, pod NotReady
Recovery Time	20–90 min depending on data state

Symptoms

- kubectl get pods shows a PostgreSQL pod with status CrashLoopBackOff and a high restart count.
- Pod logs show PostgreSQL failing to start: 'invalid data in pg_control' or 'Permission denied'.
- Patroni cannot register the pod as a cluster member.
- PVC is attached but PostgreSQL cannot read or write the data directory.
- Percona operator keeps re-creating the pod in an endless restart loop.

⌨ DIAGNOSTIC COMMANDS

# — Crash reason from logs	
Previous crash logs (postgres)	`kubectl logs -n <ns> <pg-pod> -c postgres --previous --tail=150`
Previous crash logs (patroni)	`kubectl logs -n <ns> <pg-pod> -c patroni --previous --tail=100`
Init container logs	`kubectl logs -n <ns> <pg-pod> -c pgbackrest-restore --previous 2>/dev/null || echo 'no restore container'`
Pod events	`kubectl describe pod -n <ns> <pg-pod> | grep -A15 Events`

# — Data directory investigation via debug pod	
Attach debug pod to PVC	`kubectl debug -n <ns> <pg-pod> -it --image=postgres:16 --share-processes -- bash`
Check data dir permissions	ls -la /pgdata/
pg_control validation	`pg_controldata /pgdata/ 2>&1 | head -20`

Config syntax check	postgres --config-file=/pgdata/postgresql.conf --check 2>&1

> ⚠ **WARNING**
> pg_resetwal destroys WAL history and can cause data loss. Only use as an absolute last resort after all other options have been exhausted and a recovery from backup has been confirmed not possible.

Resolution Steps

1	Read the previous container logs — the crash cause is almost always explicit (permission denied, bad config syntax, or 'invalid data in pg_control').
2	For permission issues: /pgdata must be owned by the postgres user (UID 26 in Percona images). Add an initContainer with: chown -R 26:26 /pgdata.
3	For postgresql.conf syntax errors: mount a debug pod, fix the ConfigMap entry, and redeploy — or exec into a running pod if it briefly starts.
4	For pg_control corruption: first attempt restore from the latest pgBackRest backup via PerconaPGRestore CR.
5	For init container (pgBackRest restore) failure: check init container logs and pgBackRest configuration — the stanza or S3 credentials may be wrong.
6	After resolving root cause, delete the pod to trigger a clean restart: kubectl delete pod <pg-pod> -n <ns>.
7	Confirm the pod reaches Running state and Patroni registers it in the cluster: patronictl list.

ISSUE 17 — Resource Quota / LimitRange Blocking Pod Startup

MEDIUM

Component	Kubernetes ResourceQuota / LimitRange
Affected Layer	Resource Scheduling
Common Cause	Namespace quota exceeded, missing resource requests on PostgreSQL pods, LimitRange defaults too restrictive
Alert Name	Pod pending: 'exceeded quota', scheduler FailedCreate event
Recovery Time	15–30 min

Symptoms

- New PostgreSQL pods stuck in 'Pending' state indefinitely.
- kubectl describe pod shows: 'exceeded quota: namespace has quota, resource cpu/memory insufficient'.
- Percona operator cannot scale up the cluster when updating the CR replica count.
- kubectl get events shows 'FailedCreate' for the StatefulSet.
- LimitRange admission webhook blocking pod creation due to missing resource limits in the CR.

⌨ DIAGNOSTIC COMMANDS

— Quota and resource investigation

Namespace quota status	`kubectl describe resourcequota -n <ns>`
LimitRange rules	`kubectl get limitrange -n <ns> -o yaml`
Namespace resource summary	`kubectl describe namespace <ns>`

— Pod scheduling failure

Pending pod events	`kubectl describe pod -n <ns> <pending-pod> | grep -A10 Events`
StatefulSet	`kubectl describe statefulset -n <ns> <sts-name> | grep -A10 Events`

events	
Pod resource requests	`kubectl get pod -n <ns> <pg-pod> -o jsonpath='{.spec.containers[*].resources}'`

```
# — Node capacity check
```

Node allocated resources	`kubectl describe nodes | grep -A8 'Allocated resources'`
Pod CPU/memory usage	`kubectl top pod -n <ns> --sort-by=memory`

Resolution Steps

1	Identify which resource is exhausted (CPU, memory, or pod count) via kubectl describe resourcequota.
2	If the quota is legitimately full due to other workloads, increase it: kubectl edit resourcequota <quota-name> -n <ns>.
3	If the PostgreSQL pod's resource requests are missing, set them in the PerconaPGCluster CR under spec.instances[*].resources.requests and spec.instances[*].resources.limits.
4	If LimitRange is applying defaults that are too low for PostgreSQL, update the LimitRange max values: kubectl edit limitrange <lr-name> -n <ns>.
5	If the node itself is out of capacity (not a quota issue), add a new Alma9 VM node to the Kubernetes cluster.
6	After quota increase, watch for the pod to start: kubectl get pod -n <ns> -w.

ISSUE 18 — DNS / Service Discovery Failures — MEDIUM

Component	Kubernetes DNS / CoreDNS / Services
Affected Layer	Service Discovery
Common Cause	CoreDNS crash or overload, stale DNS cache, headless service misconfiguration, NDOTS setting
Alert Name	Application NXDOMAIN errors, CoreDNS error rate alert
Recovery Time	10–30 min

Symptoms

- Applications fail with 'could not translate host name to address' or NXDOMAIN errors.
- Intermittent connection failures that resolve on retry (transient DNS resolution failures).
- kubectl exec into app pod: nslookup <pg-service> fails or takes > 5 seconds.
- CoreDNS pods show high CPU or memory usage.
- HAProxy health checks fail intermittently due to DNS lookup failures for backend pods.

⌨ DIAGNOSTIC COMMANDS

# — DNS resolution testing	
DNS lookup from app pod	`kubectl exec -n <app-ns> <app-pod> -- nslookup <pg-svc>.<pg-ns>.svc.cluster.local`
DNS timing test	`kubectl exec -n <app-ns> <app-pod> -- dig <pg-svc>.<pg-ns>.svc.cluster.local | grep 'Query time'`
resolv.conf inside pod	`kubectl exec -n <app-ns> <app-pod> -- cat /etc/resolv.conf`
# — CoreDNS health	
CoreDNS pod status	`kubectl get pod -n kube-system -l k8s-app=kube-dns`

CoreDNS error logs	`kubectl logs -n kube-system <coredns-pod> --tail=100 | grep -i error`
CoreDNS ConfigMap	`kubectl get configmap -n kube-system coredns -o yaml`

```
# ── Service endpoint validation ──────────────────────────────────
```

Service endpoints	`kubectl get endpoints -n <ns> <pg-service>`
Service label selector	`kubectl describe service -n <ns> <pg-service> | grep -A5 Selector`

Resolution Steps

1	Test DNS resolution from the application pod using nslookup and dig. Identify if failures are intermittent or consistent.
2	Restart CoreDNS pods to clear any in-memory corruption: kubectl rollout restart deployment/coredns -n kube-system.
3	Verify the PostgreSQL service has active endpoints: kubectl get endpoints <pg-svc> -n <ns>. Empty endpoints mean backend pods are not passing health checks.
4	Use fully-qualified domain names (FQDNs) in application connection strings: <svc>.<ns>.svc.cluster.local to bypass NDOTS search path issues.
5	If CoreDNS is CPU-saturated, scale it out: kubectl scale deployment coredns -n kube-system --replicas=4.
6	For the Patroni headless service used by pod-to-pod communication, verify the selector labels match the actual pod labels: kubectl get pod --show-labels and compare.
7	To reduce DNS lookup volume, add dnsConfig to application pods: ndots: 2 — this reduces unnecessary search-path queries.

ISSUE 19 — Rolling Upgrade / Version Upgrade Failure — HIGH

Component	Percona Operator / PostgreSQL
Affected Layer	Lifecycle Management
Common Cause	Incompatible extensions, missing pre-upgrade steps, insufficient PDB allowance, image pull failure
Alert Name	Pods in ImagePullBackOff or Init:Error during upgrade, cluster degraded
Recovery Time	30 min to several hours for rollback

Symptoms

- During image version bump in PerconaPGCluster CR, pods stuck in Init or ImagePullBackOff.
- PostgreSQL fails to start after new image is pulled due to extension version mismatch.
- Patroni reports 'postgres is not running' after new image starts.
- PodDisruptionBudget preventing rolling restart from completing.
- Major version upgrade: data directory needs pg_upgrade but operator does not run it automatically.

⌨ DIAGNOSTIC COMMANDS

— Pre-upgrade validation

Installed extensions

```
kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \   "SELECT name, default_version, installed_version FROM pg_available_extensions WHERE installed_version IS NOT NULL;"
```

Current PDB config

```
kubectl get pdb -n <ns> -o wide
```

Current image version

```
kubectl get statefulset -n <ns> -o jsonpath='{.items[*].spec.template.spec.containers[*].image}'
```

# — During/after upgrade	
Pod image pull status	`kubectl get pod -n <ns> -w`
Init container failure	`kubectl describe pod -n <ns> <pg-pod> | grep -A20 'Init Containers'`
Operator upgrade events	`kubectl get events -n <ns> --sort-by=.metadata.creationTimestamp | tail -30`
# — Rollback	
Rollback image in CR	`kubectl patch perconapgcluster <name> -n <ns> --type=merge \ -p '{"spec":{"image":"percona/percona-postgresql-operator:2.x.y-ppg16.previous"}}'`

Resolution Steps

1	Take a full pgBackRest backup before any upgrade: apply a PerconaPGBackup CR and wait for completion.
2	Verify the new container image is accessible from all nodes: kubectl run test --image=<new-image> --restart=Never -- echo ok.
3	If pods show ImagePullBackOff: verify the image tag exists in your registry and imagePullSecret is configured correctly in the CR.
4	If PostgreSQL crashes after image update due to extension version mismatch: immediately rollback the image tag in the PerconaPGCluster CR to the previous version.
5	For major version upgrades (e.g., PG15 → PG16): Percona operator does NOT run pg_upgrade automatically. Use a dump/restore approach or the pg_upgrade utility on a separate instance.
6	If PodDisruptionBudget is blocking the rolling restart: temporarily patch it — kubectl patch pdb <pdb> -n <ns> -p '{"spec":{"maxUnavailable":2}}'.
7	After upgrade completes, run ANALYZE on all databases and verify pg_stat_user_tables is healthy: SELECT count(*) FROM

```
pg_stat_user_tables WHERE last_analyze IS NULL;
```

ISSUE 20 — Logical Replication / Subscription Failure — HIGH

Component	PostgreSQL Logical Replication
Affected Layer	Data Streaming / CDC
Common Cause	Inactive slot, schema mismatch between publisher and subscriber, WAL too far behind
Alert Name	pg_replication_slots inactive with growing lag, pg_stat_subscription stale
Recovery Time	15–90 min depending on lag size

Symptoms

- Logical replication consumer (Debezium, downstream PostgreSQL) reporting no new events.
- pg_replication_slots shows active=false with growing WAL lag in bytes.
- WAL files accumulating on the primary due to the inactive slot.
- pg_stat_subscription on the subscriber shows last_msg_receipt_time stale by hours.
- Downstream data pipeline showing stale data or a broken CDC stream.

DIAGNOSTIC COMMANDS

```
# — On publisher: slot and publication status
```

Logical slot lag

```
kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \   "SELECT slot_name, plugin, active, pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),confirmed_flush_lsn)) AS lag    FROM pg_replication_slots WHERE slot_type='logical';"
```

Publications

```
kubectl exec -n <ns> <pg-pod> -- psql -U postgres -
```

	`d <db> -c \ "SELECT pubname, puballtables, pubinsert, pubupdate, pubdelete FROM pg_publication;"`
`# — On subscriber`	
Subscription status	`kubectl exec -n <ns> <sub-pod> -- psql -U postgres -c \ "SELECT subname, enabled, received_lsn, latest_end_lsn, last_msg_receipt_time FROM pg_stat_subscription;"`
Subscription errors	`kubectl exec -n <ns> <sub-pod> -- psql -U postgres -c \ "SELECT * FROM pg_stat_subscription_stats;"`
`# — Re-enable subscription`	
Enable subscription	`kubectl exec -n <ns> <sub-pod> -- psql -U postgres -c "ALTER SUBSCRIPTION <sub_name> ENABLE;"`
Refresh publication tables	`kubectl exec -n <ns> <sub-pod> -- psql -U postgres -c "ALTER SUBSCRIPTION <sub_name> REFRESH PUBLICATION;"`

Resolution Steps

1	Check if the replication slot is inactive and how large the WAL lag is — slot lag > 10 GB means the subscriber has been down for a long time.
2	Check if the subscriber can reach the publisher: psql postgresql://user:pass@<publisher-svc>:5432/db from the subscriber pod.
3	Re-enable the subscription: ALTER SUBSCRIPTION <name> ENABLE; — monitor pg_stat_subscription for received_lsn incrementing.
4	If the failure was caused by a schema change on the publisher: apply the same DDL on the subscriber first, then re-enable.
5	If WAL has been consumed (slot lag too old): drop and recreate the subscription with a fresh snapshot: ALTER SUBSCRIPTION <name> REFRESH PUBLICATION WITH (copy_data = true);
6	For Debezium/Kafka Connect: restart the connector task and verify the

	offset. Run SELECT pg_logical_emit_message(true, 'heartbeat', now()::text); on publisher to test the pipeline.
7	Monitor recovery: the slot's active flag should become true and the lag should decrease continuously in pg_replication_slots.

ISSUE 21 — Transaction ID Wraparound Emergency — CRITICAL

Component	PostgreSQL MVCC / autovacuum
Affected Layer	Data Safety
Common Cause	Autovacuum unable to run (blocked by long transaction or disabled), very high write volume
Alert Name	pg_database_age > 1.5 billion, PostgreSQL logs 'database not accepting commands'
Recovery Time	1–4 hours for emergency freeze; potential downtime required

Symptoms / Real-World Context

- REAL SCENARIO: A batch migration job runs for 18 hours holding an open transaction. autovacuum cannot advance the xmin horizon on any table. After the job finishes, pg_database_age is already at 1.8 billion. PostgreSQL begins printing 'WARNING: database must be vacuumed within 200 million transactions' every minute.
- PostgreSQL enters 'read-only' mode to prevent wraparound data corruption: 'ERROR: database is not accepting commands to avoid wraparound data loss'.
- pg_database shows age(datfrozenxid) approaching 2,100,000,000 (the hard limit before forced shutdown).
- All writes fail; reads continue; application completely down for write operations.
- Prometheus: pg_database_wraparound_age > 1500000000 fires CRITICAL alert.

⌨ DIAGNOSTIC COMMANDS

```
# — Wraparound risk assessment
```

Database XID age	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT datname, age(datfrozenxid) AS xid_age, 2100000000 - age(datfrozenxid) AS transactions_remaining FROM pg_database ORDER BY xid_age DESC;"`
Table-level freeze risk	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -d <db> -c \ "SELECT schemaname, tablename, age(relfrozenxid) AS table_age FROM pg_class JOIN pg_namespace ON relnamespace=pg_namespace.oid WHERE relkind='r' ORDER BY table_age DESC LIMIT 20;"`
Blocking transaction	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT pid, now()-xact_start AS age, state, application_name, left(query,100) FROM pg_stat_activity WHERE state != 'idle' ORDER BY xact_start LIMIT 5;"`
Active autovacuum jobs	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT pid, now()-query_start AS age, query FROM pg_stat_activity WHERE query LIKE '%autovacuum%';"`

> ⚠ **WARNING**
> If pg_database age exceeds 2,100,000,000 (2.1 billion), PostgreSQL will SHUT DOWN and refuse all connections to prevent data corruption. This requires manual VACUUM FREEZE intervention from a superuser before the database can be brought back online.

Emergency Resolution Steps

1	IMMEDIATELY terminate any long-running transactions blocking autovacuum: SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE state != 'idle' AND now()-xact_start > interval '1 hour';
2	Set aggressive autovacuum parameters for the emergency: ALTER SYSTEM SET autovacuum_freeze_max_age = 200000000; ALTER SYSTEM SET vacuum_freeze_min_age = 0; SELECT pg_reload_conf();
3	Run aggressive VACUUM FREEZE on the oldest tables first (in priority order by table_age DESC): VACUUM FREEZE VERBOSE schema.tablename;

4	Increase autovacuum workers temporarily: ALTER SYSTEM SET autovacuum_max_workers = 8; SELECT pg_reload_conf();
5	Monitor progress: SELECT datname, age(datfrozenxid) FROM pg_database ORDER BY age DESC; — the age should decrease as VACUUM FREEZE progresses.
6	If PostgreSQL has already entered read-only mode: connect as a superuser and run VACUUM FREEZE on each table manually — this is the only operation permitted in this mode.
7	After age drops below 500 million, restore autovacuum_freeze_max_age to the production value (200000000) and validate autovacuum is running normally.
8	Post-incident: identify the root cause (long-running transaction pattern) and implement idle_in_transaction_session_timeout = 1h in postgresql.conf to prevent recurrence.

Prevention

- Alert at age > 750M (WARN), > 1.2B (HIGH), > 1.5B (CRITICAL) — give yourself time to act.
- Set idle_in_transaction_session_timeout = '3600s' to auto-kill abandoned transactions.
- Use statement_timeout = '4h' on batch job database roles to prevent runaway queries.
- Monitor pg_database_age Prometheus metric daily and include it in weekly health reports.

ISSUE 22 — Split-Brain After Network Partition Recovery — CRITICAL

Component	Patroni / PostgreSQL
Affected Layer	HA / Data Integrity

Common Cause	Network partition between Kubernetes nodes isolates leader; both sides promote a leader; partition heals with two primaries
Alert Name	patroni_master_is_running on multiple nodes, pg_replication lag diverging
Recovery Time	1–3 hours including data reconciliation

Symptoms / Real-World Context

- REAL SCENARIO: The Kubernetes cluster spans two VM hypervisor hosts connected via a 10GbE trunk. A switch misconfiguration causes a 7-minute network partition. Node1 (primary) continues receiving writes. Node2 and Node3 (replicas) cannot reach etcd or Node1 and elect Node2 as a new primary. When the partition heals, both Node1 and Node2 are accepting writes — classic split-brain.
- Patroni detects the conflict on partition heal and demotes the node with the older DCS lock.
- The demoted node's uncommitted transactions are gone; applications may have received success acknowledgements for writes that are now lost.
- pg_stat_replication shows significant LSN divergence between the two former primaries.
- Application-level data inconsistency: orders, transactions, or records created during the split-brain window may be missing.

⌨ DIAGNOSTIC COMMANDS

# — Identify divergence	
Current LSN on each node	`for pod in <pod1> <pod2> <pod3>; do echo "--- $pod ---"; kubectl exec -n <ns> $pod -- psql -U postgres -c 'SELECT pg_current_wal_lsn();'; done`
Patroni cluster state	`kubectl exec -n <ns> <any-pod> -- patronictl -c /etc/patroni/patroni.yml list`
Replication timeline	`kubectl exec -n <ns> <pg-pod> -- patronictl -c /etc/patroni/patroni.yml history`
# — Check for timeline divergence	

pg_controldata timeline	`kubectl exec -n <ns> <pg-pod> -- pg_controldata /pgdata/ | grep -i timeline`
WAL timeline files	`kubectl exec -n <ns> <pg-pod> -- ls -la /pgdata/pg_wal/*.history 2>/dev/null`

> ⚠ **WARNING**
> In a split-brain scenario, you must decide which node is the authoritative primary. This is a DATA INTEGRITY decision that requires input from the application team. Writes made to the 'wrong' primary during the partition may need to be replayed, discarded, or merged depending on your data model.

Resolution Steps

1	Patroni usually resolves split-brain automatically on partition heal by demoting the node with the older etcd lock. Verify this has happened: patronictl list should show exactly one Leader.
2	Identify the divergence window: compare pg_current_wal_lsn() on both former primaries and determine how many transactions were written to the 'demoted' primary.
3	Work with the application team to audit writes during the divergence window. Query application logs for transactions between partition-start and partition-heal timestamps.
4	If the demoted primary had unique writes (e.g., orders or payments), extract them: pg_waldump /pgdata/pg_wal/<diverged-wal-segment> and replay them on the new primary.
5	Reinitialize the demoted node as a replica from the current primary: patronictl reinit <cluster> <node> --force.
6	Conduct a data integrity check: row counts, max IDs, and checksums on critical tables on both the primary and all replicas.
7	After recovery: implement synchronous_commit = remote_write and synchronous_standby_names to prevent split-brain writes in future (at the cost of slight latency).

8	Review the network partition root cause and implement redundant network paths on the Alma9 VM hypervisor layer.

ISSUE 23 — pgBackRest Restore Taking Too Long / Hung — HIGH

Component	pgBackRest / PerconaPGRestore CR
Affected Layer	Backup & Recovery
Common Cause	Large backup size, network bandwidth to S3 throttled, restore WAL replay bottleneck, wrong restore options
Alert Name	PerconaPGRestore CR stuck in 'restoring' state > 2 hours, RTO breached
Recovery Time	Varies: 30 min to 24+ hours depending on backup size and network

Symptoms / Real-World Context

- REAL SCENARIO: A production cluster suffers data corruption due to a bad migration. A restore is initiated via PerconaPGRestore CR. 4 hours later the restore pod is still running and the database is not available. The team has no visibility into restore progress or estimated completion time.
- PerconaPGRestore CR status shows 'restoring' indefinitely.
- Restore pod is running but logs show low throughput or repeated retries.
- Network monitoring on the Alma9 node shows very low or intermittent I/O to S3.
- WAL replay phase is running but progress is not visible.

⌨ DIAGNOSTIC COMMANDS

```
# — Restore status
```

Restore CR status	`kubectl get perconapgrestore -n <ns> -o wide`

Restore CR detail	`kubectl describe perconapgrestore -n <ns> <restore-name>`
Restore pod logs	`kubectl logs -n <ns> <pg-pod> -c pgbackrest-restore --follow`

```
# ── pgBackRest restore progress ──────────────────────────────────────
```

pgBackRest info during restore	`kubectl exec -n <ns> <pg-pod> -- pgbackrest info --stanza=<stanza>`
Restore process on node	`ssh <node-ip> 'ps aux | grep pgbackrest' ssh <node-ip> 'iostat -x 2 5' ssh <node-ip> 'ss -tn | grep :443'`

```
# ── WAL replay progress (PostgreSQL recovery mode) ──────────────────
```

Recovery progress	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT pg_is_in_recovery(), pg_last_wal_replay_lsn(), pg_last_wal_receive_lsn(), now()-pg_last_xact_replay_timestamp() AS replay_lag;"`

Resolution Steps

1	Check pgBackRest restore logs for the current phase: 'restore backup', 'build/link tablespaces', or 'replay WAL'. Each phase has different performance characteristics.
2	If the restore is progressing slowly due to S3 bandwidth throttling: check if your S3 endpoint has rate limits or if the node has network ACLs limiting outbound bandwidth.
3	Increase pgBackRest parallelism for the restore: pgbackrest restore --stanza=<name> --process-max=8 (if restarting manually).
4	If stuck in WAL replay phase: check recovery.signal or standby.signal file exists in /pgdata, and that recovery parameters are correct in postgresql.auto.conf.
5	To monitor WAL replay progress, the query SELECT now()-pg_last_xact_replay_timestamp() AS lag should decrease over time if WAL

	is being applied.
6	If the restore appears hung with no activity for > 30 minutes: cancel it (delete the restore pod), investigate the error in logs, and retry with correct parameters.
7	Conduct a post-restore RTO analysis and document actual restore time vs your recovery time objective to identify gap and tuning opportunities.

Prevention & RTO Planning

- Benchmark restore time quarterly: restore to a test namespace and measure time for your backup set size.
- Use pgBackRest --delta restore for incremental restores when the data directory already exists.
- Configure repo-s3-upload-chunk-size and process-max in pgBackRest for parallel multi-part uploads/downloads.
- Document estimated restore time (RTO) in the runbook based on backup size and measured network throughput.
- Consider pgBackRest standby restore (warm standby) to reduce RTO by keeping a partially-restored replica ready.

ISSUE 24 — StatefulSet Volume Claim Template Changes Blocked

MEDIUM

Component	Kubernetes StatefulSet / Percona Operator
Affected Layer	Storage Configuration
Common Cause	StatefulSet VolumeClaimTemplates are immutable; storage size increase requires manual steps
Alert Name	Operator reconcile error: 'forbidden: updates to statefulset spec for fields other than...'
Recovery Time	30–60 min including data migration steps

Symptoms / Real-World Context

- REAL SCENARIO: The DBA team increases the PostgreSQL storage from 200 Gi to 500 Gi in the PerconaPGCluster CR. The Percona operator logs show 'cannot patch statefulset: forbidden: updates to statefulset spec for fields other than replicas, template and updateStrategy are not permitted'. The storage size does not change.
- Percona operator repeatedly logs the same reconcile error but takes no action.
- kubectl describe statefulset shows the old storage size in VolumeClaimTemplates.
- Manually editing the StatefulSet is rejected by the Kubernetes API with a 422 Unprocessable Entity error.
- Existing PVCs remain at the old size; pods continue to run on the old volumes.

DIAGNOSTIC COMMANDS

```
# — Investigate the immutability constraint
```

Operator error logs	`kubectl logs -n <ns> deploy/percona-postgresql-operator --tail=100 | grep -i 'forbidden|immutable|statefulset'`
StatefulSet VCT size	`kubectl get statefulset -n <ns> <sts-name> \ -o jsonpath='{.spec.volumeClaimTemplates[*].spec.resources.requests.storage}'`
PVC current sizes	`kubectl get pvc -n <ns>`
Storage class expandable	`kubectl get storageclass <sc-name> \ -o jsonpath='{.allowVolumeExpansion}'`

```
# — Manual PVC expansion (if StorageClass allows it)
```

Patch individual PVC	`kubectl patch pvc <pvc-name> -n <ns> --type=merge \ -p '{"spec":{"resources":{"requests":{"storage":"500Gi"}}}}'`
Watch PVC resize status	`kubectl get pvc -n <ns> -w`

Resolution Steps

1	Expand PVCs directly (bypassing the StatefulSet immutability): kubectl patch pvc <pvc-name> -n <ns> ... for each pod's PVC. This works if the StorageClass has allowVolumeExpansion: true.
2	Delete the StatefulSet WITHOUT deleting pods (--cascade=orphan): kubectl delete statefulset <sts-name> -n <ns> --cascade=orphan. Existing pods and PVCs are preserved.
3	Update the PerconaPGCluster CR with the new storage size. The Percona operator will recreate the StatefulSet with the updated VolumeClaimTemplates.
4	Verify the StatefulSet is recreated with the new storage spec: kubectl describe statefulset <sts-name> -n <ns>.
5	New pods created by scale-up or pod replacement will use the new larger PVC size. Existing pods' PVCs are already expanded in step 1.
6	Verify Patroni cluster is healthy after the StatefulSet recreation: patronictl list should show all members in sync.

ℹ️ NOTE
In Kubernetes, StatefulSet VolumeClaimTemplates are immutable by design. You cannot change the storage size by patching the StatefulSet directly. The delete-and-recreate pattern (with --cascade=orphan) is the standard approach. Always expand existing PVCs first before recreating the StatefulSet.

ISSUE 25 — High WAL Generation Rate Causing I/O Saturation

HIGH

Component	PostgreSQL WAL Writer
Affected Layer	Storage I/O / Replication
Common Cause	Unoptimised UPDATE patterns (full-row rewrites), lack of HOT updates, TOAST updates, bulk loads

Alert Name	pg_wal_files_count > 500, node disk I/O utilisation > 90%
Recovery Time	30–120 min for tuning; immediate mitigation via throttling

Symptoms / Real-World Context

- REAL SCENARIO: After a feature deployment, an application begins updating a 'last_seen' timestamp column on every API request (millions per hour). Each UPDATE rewrites the entire row in WAL even if only one column changes. WAL generation jumps from 5 GB/hour to 80 GB/hour. Disk I/O on the Alma9 VM node saturates. Replication lag climbs to 120 seconds.
- iostat on the node shows 100% disk utilisation on the PVC backing device.
- pg_stat_bgwriter shows checkpoints happening much more frequently than checkpoint_completion_target allows.
- Replication lag increases proportionally with WAL generation rate.
- pgBackRest WAL archiving falls behind; the archive queue grows continuously.

⌨ DIAGNOSTIC COMMANDS

— WAL generation analysis

WAL generation rate	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), '0/0'::pg_lsn) - lag(pg_wal_lsn_diff(pg_current_wal_lsn(),'0/0'::pg_lsn)) OVER ()) FROM generate_series(1,2) LIMIT 1;"`
Tables generating most WAL	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -d <db> -c \ "SELECT relname, n_tup_upd, n_tup_hot_upd, round(n_tup_hot_upd::numeric/greatest(n_tup_upd,1)*100,1) AS hot_pct FROM pg_stat_user_tables ORDER BY n_tup_upd DESC LIMIT 20;"`
Checkpoint frequency	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT checkpoints_timed, checkpoints_req, buffers_checkpoint FROM pg_stat_bgwriter;"`
Node I/O utilisation	`ssh <node-ip> 'iostat -x 2 10 | grep -E "Device|sd"'`

Resolution Steps

1	Identify the tables with the lowest HOT update ratio (n_tup_hot_upd / n_tup_upd < 0.5). These generate the most WAL because full row rewrites are happening instead of heap-only tuples.
2	For HOT updates to work, the updated columns must NOT be part of any index, and there must be free space on the heap page (fillfactor < 100). Reduce fillfactor: ALTER TABLE <t> SET (fillfactor=70);
3	Enable WAL compression to reduce WAL volume without changing application behaviour: ALTER SYSTEM SET wal_compression = on; SELECT pg_reload_conf();
4	For bulk load operations (ETL, data migrations): disable WAL logging during the bulk load phase using UNLOGGED tables, then convert back: ALTER TABLE <t> SET LOGGED;
5	Review and fix application-level anti-patterns: avoid updating rows if the data has not actually changed; use conditional updates: UPDATE t SET col=val WHERE col != val;
6	Tune checkpoint settings to reduce I/O spikes: ALTER SYSTEM SET checkpoint_completion_target = 0.9; max_wal_size = '4GB'; min_wal_size = '1GB';
7	If the WAL rate is an ongoing business requirement, upgrade the Alma9 VM node to use NVMe SSD storage for the PVC backing device.

Prevention

- Monitor n_tup_hot_upd / n_tup_upd ratio for all high-write tables. HOT ratio < 50% is a bloat and WAL amplification risk.
- Set fillfactor = 70-80 on tables with frequent single-column updates to enable HOT updates.
- Alert: pg_stat_bgwriter.checkpoints_req > checkpoints_timed — indicates WAL being generated faster than the checkpoint cycle.
- Review application ORM patterns quarterly; UPDATE *=* patterns are the most common cause of WAL amplification in web applications.

ISSUE 26 — Kubernetes Secret Drift — PostgreSQL Config Out of Sync

MEDIUM

Component	Kubernetes Secrets / ConfigMaps / Percona Operator
Affected Layer	Configuration Management
Common Cause	Manual Secret edits bypassing operator, external-secrets sync failure, Vault lease expiry, operator ignoring external changes
Alert Name	Authentication failures spike, Patroni configuration mismatch between nodes
Recovery Time	20–40 min

Symptoms / Real-World Context

- REAL SCENARIO: The security team rotates the PostgreSQL superuser password in HashiCorp Vault. The Kubernetes external-secrets operator syncs the new password to the Secret. However, the Percona operator does not detect the Secret change and does not update pgbouncer's userlist.txt or Patroni's superuser credentials. PGBouncer continues using the old cached password for 30 minutes until its connection pool is exhausted, at which point all new connections begin failing.
- PGBouncer logs show authentication failures for the superuser after a credential rotation.
- Patroni fails to connect to the leader for health checks after the postgres user password changes.
- kubectl describe perconapgcluster shows no recent reconcile triggered after the Secret was updated.
- Different pods in the cluster are using different versions of credentials (some cached, some new).

⌨ DIAGNOSTIC COMMANDS

```
# — Secret and ConfigMap audit
```

List all	`kubectl get secret -n <ns> | grep -i postgres`

PostgreSQL secrets	
Decode superuser secret	`kubectl get secret -n <ns> <pg-superuser-secret> \ -o jsonpath='{.data.password}' | base64 -d`
Check secret last modified	`kubectl get secret -n <ns> <pg-secret> \ -o jsonpath='{.metadata.creationTimestamp}'`
`# — Verify config in use by running pods`	
Patroni superuser config	`kubectl exec -n <ns> <pg-pod> -- cat /etc/patroni/patroni.yml | grep -A5 superuser`
PGBouncer userlist	`kubectl exec -n <ns> <pgb-pod> -- cat /etc/pgbouncer/userlist.txt`
external-secrets status	`kubectl get externalsecret -n <ns>`

Resolution Steps

1	Identify which component has the stale credential: check Patroni yaml, PGBouncer userlist.txt, and application connection strings against the current Secret value.
2	Force Patroni to reload its credentials by triggering a rolling restart via the operator: kubectl annotate perconapgcluster <name> -n <ns> kubectl.kubernetes.io/restartedAt=$(date +%s) --overwrite.
3	Reload PGBouncer credentials without full restart: update the userlist.txt ConfigMap and run RELOAD; in the PGBouncer admin console.
4	If using external-secrets, trigger a manual sync: kubectl annotate externalsecret <es-name> -n <ns> force-sync=$(date +%s) --overwrite.
5	After syncing, force a reconcile of the PerconaPGCluster CR: kubectl annotate perconapgcluster <name> -n <ns> reconcile=$(date +%s) --overwrite.
6	Verify all pods are using the new credentials: test connections from each pod using the rotated password.

7	Document a credential rotation runbook that includes the explicit steps for triggering operator reconcile after Secret updates.

Prevention

- Configure the Percona operator to watch specific Secrets and auto-reconcile on changes (check operator version changelog for this feature).
- Implement a post-rotation webhook that triggers a PerconaPGCluster reconcile annotation after every credential rotation.
- Use Vault Agent Injector to keep credentials in sync automatically in all pods without manual steps.
- Test credential rotation in staging quarterly as part of security compliance procedures.

ISSUE 27 — max_connections Exhaustion at PostgreSQL Level

HIGH

Component	PostgreSQL / PGBouncer
Affected Layer	Connection Management
Common Cause	PGBouncer max_db_connections too high, multiple PGBouncer instances without shared cap, direct connections bypassing PGBouncer
Alert Name	pg_stat_activity count near max_connections, 'FATAL: remaining connection slots reserved for non-replication superuser'
Recovery Time	5–20 min

Symptoms / Real-World Context

- REAL SCENARIO: The team scales PGBouncer from 1 to 3 replicas to handle more client load. Each PGBouncer instance has max_db_connections = 150, so the three instances collectively open up to 450 connections to PostgreSQL. But

max_connections is set to 300. PostgreSQL exhausts connections. Superuser connections (for DBA access) also fail. The team cannot even connect to diagnose the issue.

- Applications receive 'FATAL: remaining connection slots are reserved for non-replication superuser connections'.
- pg_stat_activity shows connection count == max_connections.
- DBA cannot connect to terminate connections because the superuser reserved slot is also saturated.
- PGBouncer logs show 'server connection error: FATAL: sorry, too many clients already'.

DIAGNOSTIC COMMANDS

```
# — Connection count investigation ————————————————
```

Total vs max connections	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT count(*) AS current, max_connections FROM pg_stat_activity, pg_settings WHERE name='max_connections';"`
Connections by client	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT application_name, count(*), state FROM pg_stat_activity GROUP BY 1,3 ORDER BY 2 DESC;"`
Connect via reserved slot	`psql 'host=<pg-svc> port=5432 user=postgres dbname=postgres options=-c\ pg_stat_activity.track=all'`

```
# — Emergency connection termination ——————————————
```

Kill idle connections	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE state='idle' AND application_name NOT LIKE 'pg%' LIMIT 50;"`
Kill idle-in-transaction	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE state='idle in transaction' AND now()-xact_start > interval '5 minutes';"`

Resolution Steps

1	Connect using the reserved superuser connection: psql 'options=-c pg_stat_activity.track=all' — this uses the reserved superuser slot separate from max_connections.
2	Immediately kill idle connections from PGBouncer or applications to free slots: SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE state='idle';
3	Calculate the correct max_db_connections per PGBouncer instance: max_db_connections_per_bouncer = (pg_max_connections - 10 superuser_slots - replication_slots) / bouncer_replicas.
4	Update each PGBouncer ConfigMap with the correct max_db_connections value and hot-reload: RELOAD; in admin console.
5	Increase PostgreSQL max_connections if the workload genuinely requires more: ALTER SYSTEM SET max_connections = 500; — requires a PostgreSQL restart.
6	Configure superuser_reserved_connections = 5 to always keep 5 slots for DBA emergency access: ALTER SYSTEM SET superuser_reserved_connections = 5;
7	Set up connection monitoring: alert when pg_stat_activity count > max_connections * 0.85 for more than 2 minutes.

ISSUE 28 — Node Affinity / Taint Preventing PostgreSQL Pod Scheduling

MEDIUM

Component	Kubernetes Scheduler / Percona Operator
Affected Layer	Pod Scheduling
Common Cause	Node taint added without toleration in CR, affinity rules too restrictive, node labels changed after cluster creation
Alert Name	Pod pending indefinitely: 'no nodes available to schedule pods'

Recovery Time	15–30 min

Symptoms / Real-World Context

- REAL SCENARIO: The infrastructure team adds a NoSchedule taint (dedicated=db:NoSchedule) to the database nodes to prevent non-database workloads. However, the PerconaPGCluster CR does not have a corresponding toleration, so the PostgreSQL pods are now unschedulable. After a rolling restart, pods go to Pending and never schedule.
- kubectl get pods shows PostgreSQL pods in Pending state for more than 5 minutes.
- kubectl describe pod shows: 'Warning: 0/5 nodes are available: 3 node(s) had untolerated taint {dedicated: db}'.
- The Percona operator does not detect scheduling failures and continues reporting the cluster as 'reconciling'.
- Other workloads scheduled on the same node are unaffected (they have tolerations).

⌨ DIAGNOSTIC COMMANDS

```
# ── Scheduling failure diagnosis ──────────────────────────────
```

Pending pod events	`kubectl describe pod -n <ns> <pending-pod> | grep -A15 Events`
Node taints	`kubectl get nodes -o custom-columns='NAME:.metadata.name,TAINTS:.spec.taints'`
Node labels	`kubectl get nodes --show-labels`

```
# ── CR scheduling configuration ──────────────────────────────
```

Current tolerations in CR	`kubectl get perconapgcluster -n <ns> <cluster> \ -o jsonpath='{.spec.instances[0].tolerations}'`
Current affinity in CR	`kubectl get perconapgcluster -n <ns> <cluster> \ -o jsonpath='{.spec.instances[0].affinity}'`

```
# ── Scheduler decision (dry run) ──────────────────────────────
```

Why pod is unschedulable	`kubectl get events -n <ns> --field-selector reason=FailedScheduling`

Resolution Steps

1	Confirm the exact taint key, value, and effect on the target nodes: kubectl get nodes -o json | jq '.items[].spec.taints'.
2	Add the corresponding toleration to the PerconaPGCluster CR under spec.instances[*].tolerations and spec.proxy.pgBouncer.tolerations.
3	If using node affinity, ensure the nodeAffinity.requiredDuringSchedulingIgnoredDuringExecution rules match the current node labels exactly — label changes may break existing affinity rules.
4	Apply the CR change: kubectl apply -f perconapgcluster.yaml. The operator will update the StatefulSet pod template and trigger a rolling restart.
5	Watch the pods schedule onto the correct nodes: kubectl get pod -n <ns> -o wide -w — verify the node names match the dedicated database nodes.
6	If pods still fail to schedule after adding tolerations: check anti-affinity rules that might prevent co-location on a node: kubectl describe pod shows 'node(s) didn't match Pod's anti-affinity rules'.

CR Toleration and Affinity Example

```
# PerconaPGCluster CR scheduling section
```

Toleration example	spec:
	  instances:
	  - tolerations:
	  - key: dedicated
	  operator: Equal
	  value: db
	  effect: NoSchedule

Node affinity example	affinity:
	  nodeAffinity:
	  requiredDuringSchedulingIgnoredDuringExecution:

```
      nodeSelectorTerms:
       - matchExpressions:
          - key: node-role
            operator: In
            values: [database]
```

ISSUE 29 — PostgreSQL Index Corruption Detected — CRITICAL

Component	PostgreSQL Storage / Indexes
Affected Layer	Data Integrity
Common Cause	Unclean shutdown, disk I/O error, kernel bug, out-of-order page write, filesystem bug on Alma9 node
Alert Name	pg_dump ERROR: index tuple exceeds maximum size, amcheck reports errors
Recovery Time	1–4 hours depending on index size and table criticality

Symptoms / Real-World Context

- REAL SCENARIO: After a forced VM power-cycle (caused by a hypervisor host crash), a PostgreSQL node restarts. One of the high-traffic tables starts returning 'ERROR: invalid page in block XXX of relation base/YYYYY/ZZZZZ' for some queries. pg_dump of the affected table fails. The index on the orders table is corrupted, but the underlying heap data is intact.
- Queries on a specific table return 'ERROR: invalid page in block' or 'index row size exceeds btree version limit'.
- pg_dump fails with 'ERROR: could not read block' for the affected relation.
- amcheck extension reports structural problems in B-tree indexes.
- Some queries return different result counts with and without an index scan (SET enable_indexscan=off changes results).

⌨ DIAGNOSTIC COMMANDS

# — Index integrity verification (amcheck extension required)	
Install amcheck	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -d <db> -c 'CREATE EXTENSION IF NOT EXISTS amcheck;'`
Verify B-tree index	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -d <db> -c \ "SELECT bt_index_check(index=>'<schema.index_name>'::regclass, heapallIndexed=>true);"`
Verify all indexes on table	`kubectl exec -n <ns> <pg-pod> -- psql -U postgres -d <db> -c \ "SELECT indexrelid::regclass FROM pg_index WHERE indrelid='<schema.table>'::regclass;" # Then run bt_index_check for each returned index name`
# — Disk integrity on Alma9 node	
Disk errors in kernel log	`ssh <node-ip> 'dmesg | grep -iE "I/O error|EXT4|XFS|corruption" | tail -30'`
Filesystem check	`ssh <node-ip> 'journalctl | grep -iE "filesystem|ext4|xfs" | tail -20'`
Disk SMART status	`ssh <node-ip> 'smartctl -a /dev/sda | grep -E "Reallocated|Pending|Offline|Health"'`

Resolution Steps

1	Run amcheck's bt_index_check on all indexes on the affected table to confirm the exact scope of corruption.
2	If only the index is corrupted (heap data is fine): drop and rebuild the index concurrently to avoid locking: DROP INDEX CONCURRENTLY bad_index; CREATE INDEX CONCURRENTLY new_index ON schema.table (column);
3	If heap data (the table itself) is also corrupted: stop writes to the affected table and restore the affected relation from the latest pgBackRest backup using pg_restore with --table flag.
4	Check disk health on the Alma9 VM node: run smartctl -a /dev/sdX and review dmesg for I/O errors. If disk errors are found, migrate data to a new VM before continuing.

5	After rebuilding the index, validate with amcheck again: bt_index_check should complete without errors.
6	Enable checksums on the PostgreSQL cluster if not already enabled (requires initdb or pg_upgrade): select data_checksums from pg_control_system(); — this detects future corruption early.
7	Monitor pg_stat_user_indexes for any unusual index scan error rates after recovery.

Prevention

- Enable data_checksums at cluster creation time (--data-checksums flag in initdb). This is the single most effective measure against silent data corruption.
- Run pg_amcheck on all databases weekly via a cron job or PerconaPGBackup post-action script.
- Configure PostgreSQL: wal_sync_method = fsync, fsync = on — never disable these in production.
- Use ECC RAM on all database Alma9 VM nodes to prevent bit-flip memory corruption propagating to storage.
- Alerting: monitor for 'invalid page' errors in PostgreSQL logs via Prometheus log exporter.

ISSUE 30 — Cross-Namespace Service Access Denied (NetworkPolicy Misconfiguration) — MEDIUM

Component	Kubernetes NetworkPolicy / Calico / Cilium
Affected Layer	Network Security
Common Cause	Default-deny policy blocking cross-namespace traffic, missing egress rules, namespace label selector mismatch
Alert Name	Application connection timeouts to PostgreSQL, connection refused from app namespace

Recovery Time	15–30 min

Symptoms / Real-World Context

- REAL SCENARIO: The team separates the application (app-prod namespace) and database (db-prod namespace) into different namespaces for RBAC isolation. A default-deny NetworkPolicy is applied to both namespaces. After the separation, the application cannot connect to PostgreSQL. Ping and curl from the app pod to the PGBouncer service time out. The same app could connect successfully when both were in the same namespace.
- Applications receive 'Connection timed out' (not refused) when connecting to PGBouncer.
- kubectl exec from the app pod shows nc -zv <pgbouncer-svc> 6432 hanging indefinitely.
- PGBouncer logs show no connection attempts arriving from the application.
- kubectl describe networkpolicy shows a default-deny rule with no cross-namespace exception.

⌨ DIAGNOSTIC COMMANDS

```
# — Connectivity testing
```

Test TCP from app pod	`kubectl exec -n <app-ns> <app-pod> -- \ nc -zv <pgbouncer-svc>.<db-ns>.svc.cluster.local 6432`
DNS resolution test	`kubectl exec -n <app-ns> <app-pod> -- \ nslookup <pgbouncer-svc>.<db-ns>.svc.cluster.local`

```
# — NetworkPolicy inspection
```

Policies in db namespace	`kubectl get networkpolicy -n <db-ns> -o yaml`
Policies in app namespace	`kubectl get networkpolicy -n <app-ns> -o yaml`
App namespace labels	`kubectl get namespace <app-ns> --show-labels`

DB namespace labels	`kubectl get namespace <db-ns> --show-labels`

```
# — Calico/Cilium policy trace (if available)
```

Calico policy trace	calicoctl policy trace \
	  --src-namespace=<app-ns> --src-selector='app=myapp' \
	  --dst-namespace=<db-ns> --dst-selector='app=pgbouncer' --dst-port=6432

Cilium connectivity test	`kubectl exec -n kube-system ds/cilium -- \` `cilium-health connectivity -d`

Resolution Steps

1	Label the application namespace to enable NetworkPolicy namespace selectors: kubectl label namespace <app-ns> team=app-prod.
2	Add an ingress rule to the db-prod NetworkPolicy to allow traffic from the app-prod namespace on port 6432 (PGBouncer) and 5432 (PostgreSQL direct).
3	Add an egress rule to the app-prod NetworkPolicy to allow traffic to the db-prod namespace on ports 6432 and 5432.
4	Apply the updated NetworkPolicy and test immediately: kubectl exec -n <app-ns> <app-pod> -- nc -zv <pgbouncer-svc>.<db-ns> 6432 should succeed within 5 seconds.
5	If using Calico or Cilium, verify the policy with their diagnostic tools to confirm traffic is now allowed.
6	Document the cross-namespace access matrix and include it in the network architecture diagram.

NetworkPolicy Template for Cross-Namespace Access

```
# Allow ingress to PGBouncer from app namespace
```

Ingress NetworkPolicy	apiVersion: networking.k8s.io/v1 kind: NetworkPolicy

```
metadata:
  name: allow-app-to-pgbouncer
  namespace: db-prod
spec:
  podSelector:
    matchLabels:
      app: pgbouncer
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              team: app-prod
      ports:
        - protocol: TCP
          port: 6432
```

Appendix A — Essential kubectl Quick Reference

# — Cluster & operator overview	
All PostgreSQL pods	kubectl get pod -n <ns> -o wide
Operator logs	kubectl logs -n <ns> deploy/percona-postgresql-operator --tail=200 -f
Cluster CR status	kubectl get perconapgcluster -n <ns> -o wide
Cluster CR detail	kubectl describe perconapgcluster -n <ns> <cluster>
# — Patroni	
Cluster list	kubectl exec -n <ns> <pod> -- patronictl -c /etc/patroni/patroni.yml list
Topology view	kubectl exec -n <ns> <pod> -- patronictl -c /etc/patroni/patroni.yml topology
Controlled switchover	kubectl exec -n <ns> <pod> -- patronictl -c /etc/patroni/patroni.yml switchover <cluster>
Force failover	kubectl exec -n <ns> <pod> -- patronictl -c /etc/patroni/patroni.yml failover <cluster> --force
Reinitialize replica	kubectl exec -n <ns> <pod> -- patronictl -c /etc/patroni/patroni.yml reinit <cluster> <node> --force
# — PGBouncer admin	
Show pools	kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U pgbouncer pgbouncer -c 'SHOW POOLS;'
Show stats	kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U pgbouncer pgbouncer -c 'SHOW STATS;'
Show clients	kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U

	pgbouncer pgbouncer -c 'SHOW CLIENTS;'
Hot reload config	kubectl exec -n <ns> <pgb-pod> -- psql -p 6432 -U pgbouncer pgbouncer -c 'RELOAD;'

```
# ── PostgreSQL diagnostics ──────────────────────────────────────
```

PostgreSQL version	kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c 'SELECT version();'
Replication state	kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c 'SELECT * FROM pg_stat_replication;'
Activity overview	kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c 'SELECT count(*),state FROM pg_stat_activity GROUP BY state;'
Lock waits	kubectl exec -n <ns> <pg-pod> -- psql -U postgres -c \ "SELECT pid, now()-query_start AS age, wait_event, query FROM pg_stat_activity WHERE wait_event_type='Lock';"

```
# ── pgBackRest ──────────────────────────────────────────────────
```

Backup info	kubectl exec -n <ns> <pg-pod> -- pgbackrest info
Backup check	kubectl exec -n <ns> <pg-pod> -- pgbackrest check --stanza=<stanza>
List backup CRs	kubectl get perconapgbackup -n <ns>
List restore CRs	kubectl get perconapgrestore -n <ns>

```
# ── Node debugging ──────────────────────────────────────────────
```

Node resource usage	kubectl top nodes
Pod resource usage	kubectl top pod -n <ns> --sort-by=memory
Debug node shell	kubectl debug node/<node-name> -it --image=nicolaka/netshoot

All events in namespace	kubectl get events -n <ns> --sort-by=.metadata.creationTimestamp | tail -50

Appendix B — Prometheus Alert Rules (Production Ready)

# Copy this YAML into your Prometheus PrometheusRule CR	
No Patroni leader	expr: max(patroni_master_is_running) by (cluster) == 0 for: 1m severity: critical
Replication lag	expr: pg_replication_lag > 30 for: 2m severity: critical
PVC filling up WARN	expr: kubelet_volume_stats_used_bytes / kubelet_volume_stats_capacity_bytes > 0.85 severity: warning
PVC filling up CRIT	expr: kubelet_volume_stats_used_bytes / kubelet_volume_stats_capacity_bytes > 0.95 severity: critical
PGBouncer pool saturated	expr: pgbouncer_pools_cl_waiting > 50 for: 2m severity: critical
Backup stale	expr: time() - pgbackrest_last_completed_backup_timestamp > 86400 severity: high
Exporter down	expr: up{job='postgres-exporter'} == 0 for: 2m severity: critical
Inactive replication slot	expr: pg_replication_slots_active == 0 and pg_replication_slots_pg_wal_lsn_diff > 1073741824 for: 5m severity: critical
XID wraparound risk HIGH	expr: pg_database_wraparound_age > 1200000000 for: 5m severity: high
XID wraparound risk CRIT	expr: pg_database_wraparound_age > 1500000000 for: 1m severity: critical
Table dead tuple ratio	expr: pg_stat_user_tables_n_dead_tup / (pg_stat_user_tables_n_live_tup + 1) > 0.2 severity: warning
Node disk I/O saturation	expr: rate(node_disk_io_time_seconds_total[5m]) > 0.9 for: 5m severity: high

High WAL file count	expr: pg_wal_files_count > 500 for: 10m severity: warning
Connection saturation	expr: pg_stat_activity_count / pg_settings_max_connections > 0.85 for: 2m severity: high

Appendix C — Issue Severity & Escalation Matrix

Severity	Response Time	Escalation Path	Communication	Postmortem
CRITICAL	< 5 min page	On-call SRE → DBA Lead → Engineering VP	Incident channel + status page	Required within 48h
HIGH	< 30 min	On-call SRE → DBA on-call	Incident channel	Required within 1 week
MEDIUM	< 4 hours	SRE team → DBA team	Ticket + Slack notification	Optional (weekly review)
LOW	< 24 hours	DBA team self-assigned	Ticket only	Not required